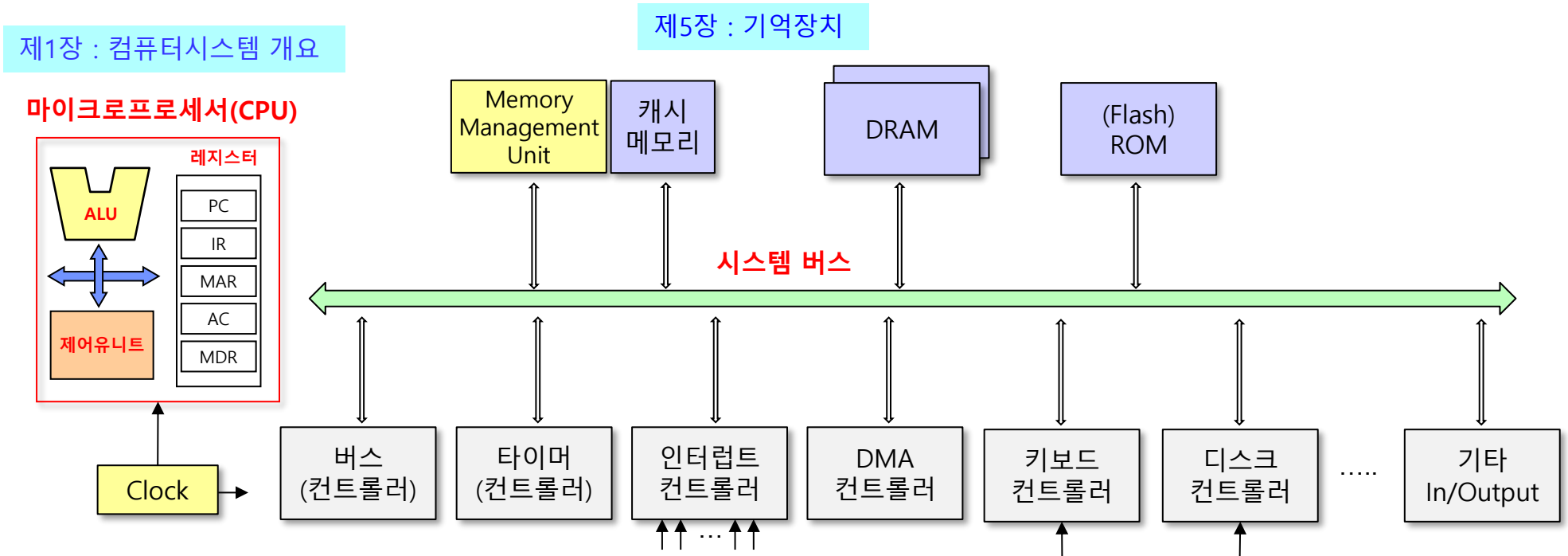
컴퓨터구조

Presented by
오 세 춘

2-3장 CPU의 구조와 기능(명령어 세트 및 주소지정 방식)

교재 주요 구성

■ 컴퓨터 시스템의 구성 – 교재 및 강의 구성 내용



배울 내용

■ 목차

- ✓ 2.1 CPU의 기본 구조
- ✓ 2.2 명령어 실행

약 1주간 학습

- ✓ 2.3 명령어 파이프라이닝

약 1주간 학습

- ✓ 2.4 명령어 세트

약 1주간 학습

  1) 연산의 종류
  2) 명령어 형식
  3) 주소지정 방식
     ① (메모리) 직접
     ② (메모리) 간접
     ③ 묵시적
     ④ 즉시
     ⑤ 레지스터 (직접)
     ⑥ 레지스터 간접
     ⑦ 변위
  4) 실제 상용 프로세서 → RISC/CISC
  5) 추가: ARM, AVR 프로세서

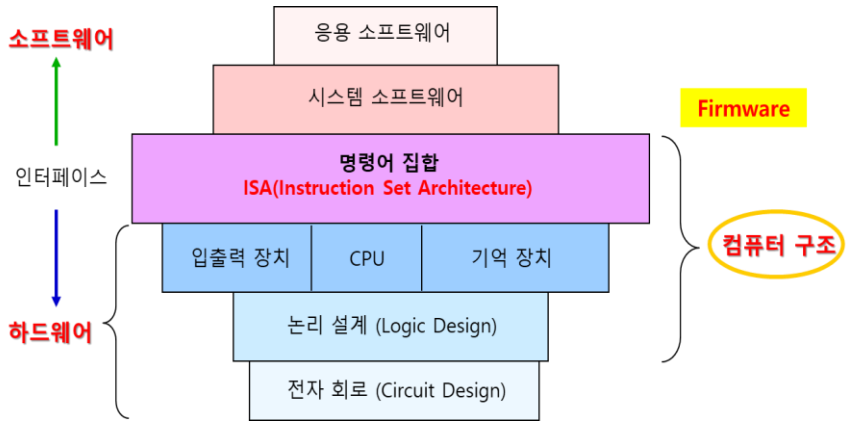
2.4 명령어 세트(instruction set)

p101

■ 명령어 세트(instruction set)

세트(set) = 집합

- ✓ 어떤 CPU를 위하여 CPU 회사에서 정의한 명령어(기계어)들의 집합
  - ➢ CPU의 설계시에 정의하고 명령어(기계어) 작성/실행시에 활용
- ✓ 명령어 세트 구조를 ISA(Instruction Set Architecture)라고 함
- ✓ 명령어 세트 설계를 위해 결정되어야 할 사항들
  1) 연산 종류(operation repertoire)
     - ❖ CPU가 수행할 연산들의 수와 종류 및 복잡도 → RISC는 명령어 종류가 적음
  2) 데이터 형태(data type)
     - ❖ 연산을 수행할 데이터들의 형태, 데이터의 길이(비트 수), 수의 표현 방식 등 →
  3) 명령어 형식(instruction format)
     - ❖ 명령어의 길이, 오퍼랜드 필드들의 수와 길이, 등 → RISC는 고정길이 명령어
  4) 주소지정 방식(addressing mode)
     - ❖ 오퍼랜드의 주소를 지정하는 방식 → RISC는 주소 지정방식의 종류가 적음



데이터 타입	8-bit CPU	16-bit CPU	32-bit CPU	64-bit CPU
char	8-bit	8-bit	8-bit	8-bit
short	16-bit	16-bit	16-bit	16-bit
int	8 or 16-bit	16-bit	32-bit	32-bit
long	32-bit	32-bit	32-bit	64-bit
long long	64-bit	64-bit	64-bit	64-bit
포인터 (Pointer)	16-bit	16-bit or 32-bit	32-bit	64-bit

보편적인 C, C++ 기준이며 프로그래밍 언어, CPU, 컴파일러 마다 다를 수 있음

2.4.1 연산의 종류

p101~p105

8086 명령어 종류

ARM 명령어 종류

■ 연산의 종류

1) 데이터 전송

- 레지스터와 레지스터 간, 레지스터와 기억장치 간, 혹은 기억장치와 기억장치 간에 데이터를 이동하는 동작

2) 산술 연산

- 덧셈, 뺄셈, 곱셈 및 나눗셈과 같은 기본적인 산술 연산들

3) 논리 연산

- 데이터의 각 비트들 간에 대한 AND, OR, NOT 및 exclusive-OR 연산

4) 입출력(I/O)

- CPU와 외부 장치들 간의 데이터 이동을 위한 동작들

5) 프로그램 제어

- 명령어 실행 순서를 변경하는 연산들
- 분기(branch), 서브루틴 호출(subroutine call)

2.4.1 연산의 종류

p101~p105

■ 서브루틴 호출을 위한 명령어들

✓ CALL 명령어

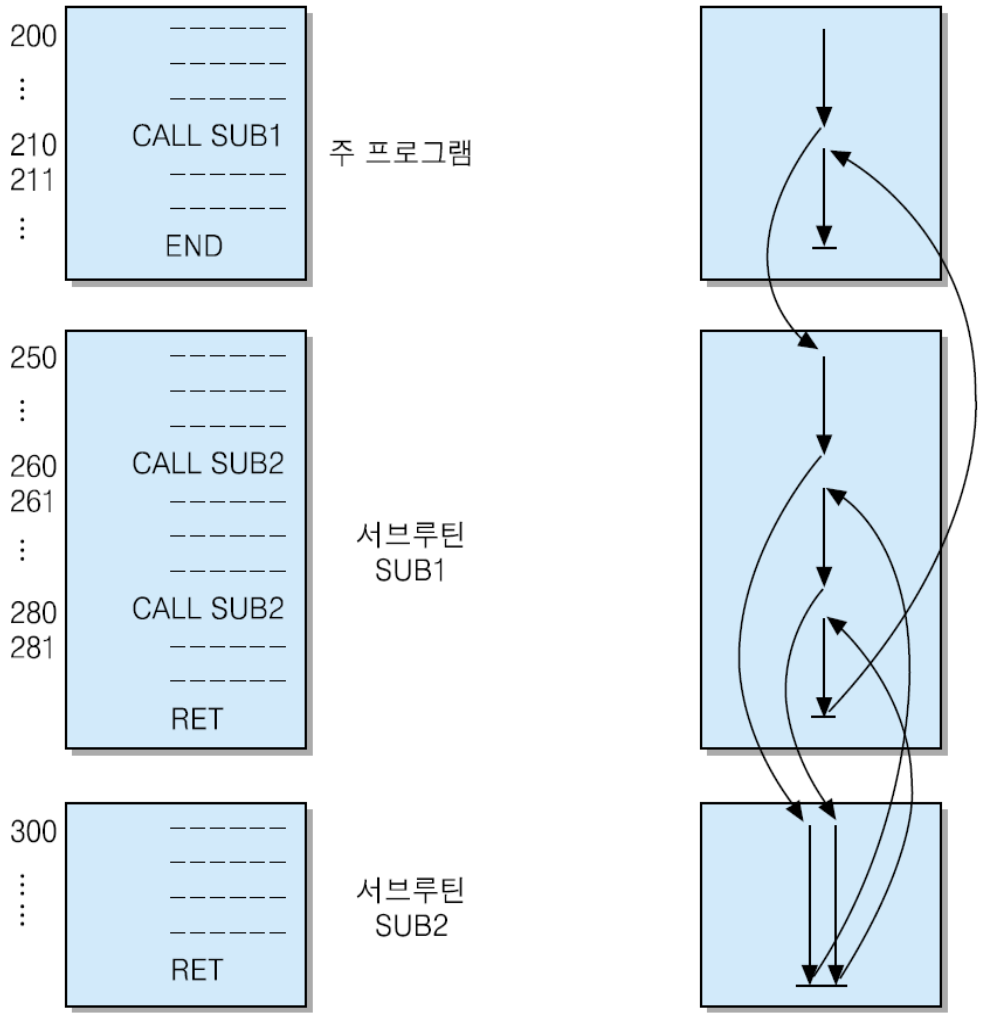
① 현재의 PC 내용을 스택에 저장하고

② 서브루틴의 시작 주소로 분기하는 명령어

✓ RET 명령어

① 원래 실행하던 프로그램으로 CPU를 복귀(return)시키는 명령어

➢ 참고: ISR(인터럽트 서비스 루틴)의 맨 마지막 줄에는 인터럽트 요청 전의 주소로 돌아가기 위해 'IRET' 명령을 사용



(a) 프로그램 구성 (b) 제어의 흐름도

2.4.1 연산의 종류

p101~p105

■ CALL/RET 명령어의 마이크로 연산

✓ CALL *X* 명령어에 대한 마이크로-연산

① t_0 : MBR ← PC

② t_1 : MAR ← SP, PC ← *X*

③ t_2 : M[MAR] ← MBR, SP ← SP - 1

➢ 현재의 PC 내용(서브루틴 수행 완료 후에 복귀할 주소)을 SP가 지정하는 스택의 최상위(top of stack)에 저장

➢ 만약 주소지정 단위가 바이트이고 저장될 주소는 16비트라면, SP ← SP – 2 로 변경

✓ RET 명령어의 마이크로-연산

① t_0: SP ← SP + 1

② t_1 : MAR ← SP

③ t_2 : PC ← M[MAR]

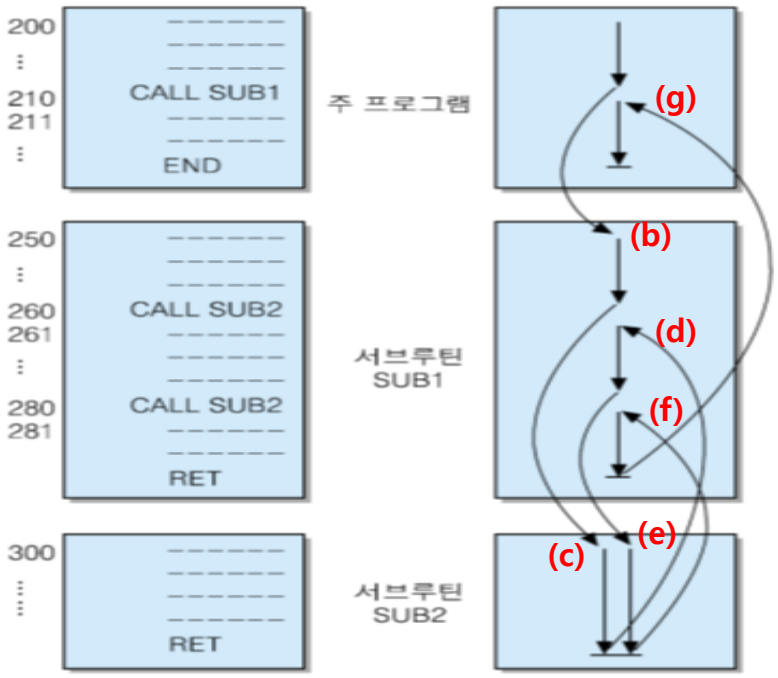
2.4.1 연산의 종류

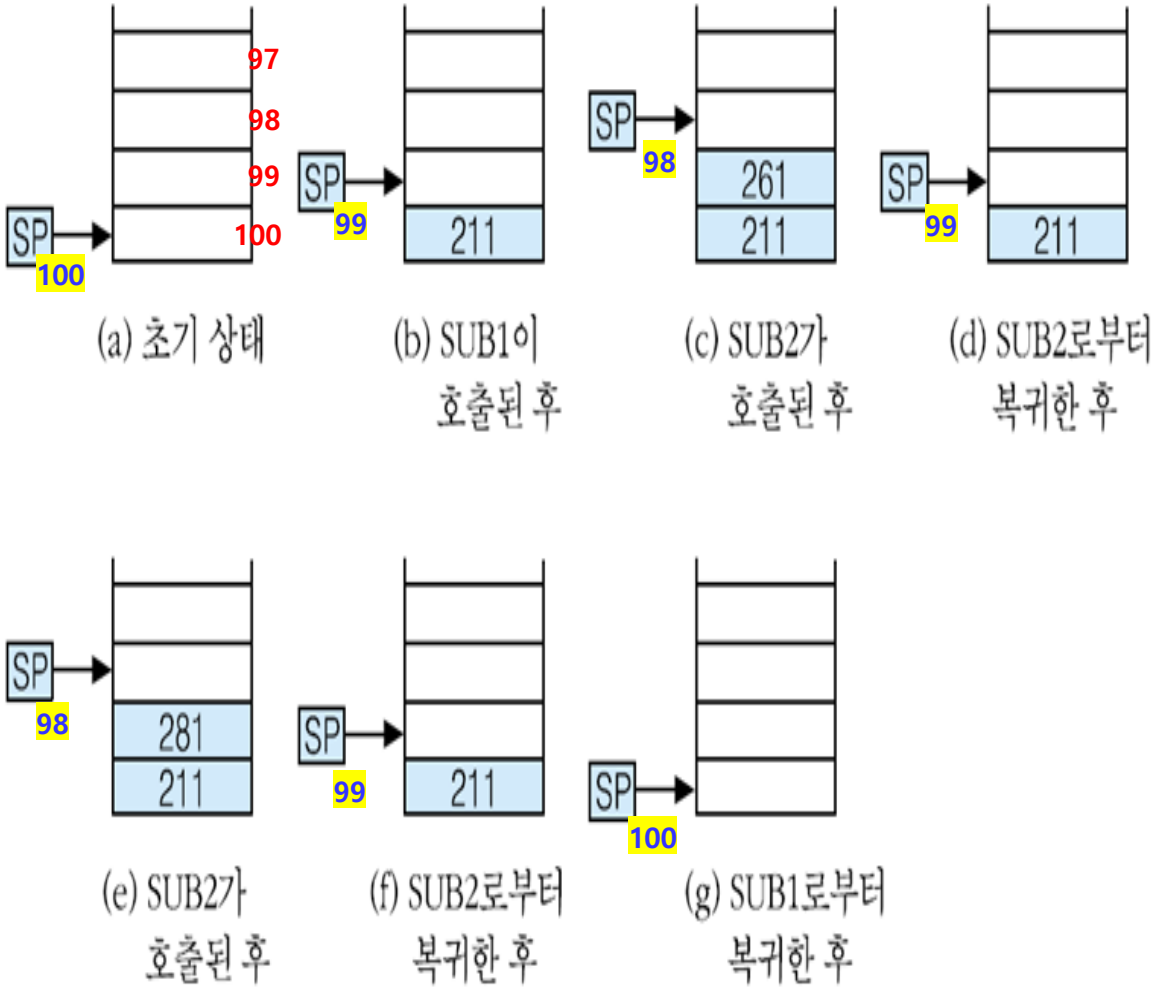
p101~p105

■ 그림 2-17의 프로그램 수행 과정에서 스택의 변화

CALL *X* 명령어에 대한 마이크로-연산
① t_0 : MBR ← PC
② t_1 : MAR ← SP, PC ← *X*
③ t_2 : M[MAR] ← MBR, SP ← SP – 1

RET 명령어의 마이크로-연산
① t_0 : SP ← SP + 1
② t_1 : MAR ← SP
③ t_2 : PC ← M[MAR]





2.4.2 명령어 형식

p106~p114

■ 명령어의 구성요소들

- ✓ 연산 코드(Operation Code)
  - ➢ 수행될 연산을 지정 (예: LOAD, ADD 등)
- ✓ 오퍼랜드(Operand)
  - ➢ 연산을 수행하는 데 필요한 데이터 혹은 데이터의 주소
  - ➢ 보통 각 연산은 한 개 혹은 두 개의 입력 오퍼랜드들과 한 개의 결과 오퍼랜드를 포함
  - ➢ 데이터는 CPU 레지스터, 주기억장치, 혹은 I/O 장치에 위치
- ✓ 다음 명령어 주소(Next Instruction Address) → 일반적으로는 교재와 다르게 오퍼랜드의 일종으로 간주
  - ➢ 현재의 명령어 실행이 완료된 후에 다음 명령어를 인출할 위치 지정
  - ➢ 분기 혹은 호출 명령어와 같이 실행 순서를 변경하는 경우에 필요

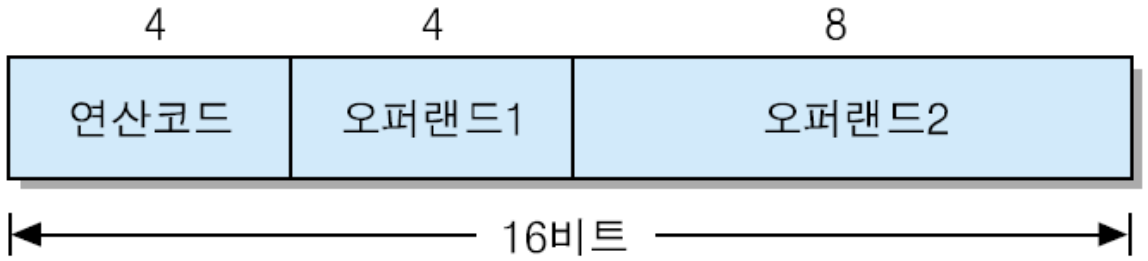
2.4.2 명령어 형식

p106~p114

■ 명령어 형식

- ✓ 명령어 형식(instruction format)
  - ➢ 명령어 내 필드들의 수와 배치 방식 및 각 필드의 비트 수
- ✓ 필드(field)
  - ➢ 명령어의 각 구성 요소들에 소요되는 비트들의 그룹
- ✓ 명령어의 길이 = 워드 길이

- ✓ [예] 세 개의 필드들로 구성된 16-비트 명령어



2.4.2 명령어 형식

p106~p114

■ 명령어 형식의 결정에서 고려할 사항들

✓ 연산 코드 필드 길이

➢ 연산의 개수를 결정

❖ [예] 4 비트 → 2^4 = 16 가지의 연산 정의 가능

➢ 만약 연산 코드 필드가 5 비트로 늘어나면, 2^5 = 32 가지 연산들 정의 가능

❖ 다른 필드의 길이가 감소

✓ 오퍼랜드 필드의 길이

➢ 오퍼랜드의 범위(크기) 결정

➢ 연산 코드 및 동일 연산코드에서도 오퍼랜드의 종류에 따라 범위가 달라짐

❖ 데이터 → 표현 가능한 수의 범위 결정

❖ 기억장치 주소 → CPU가 오퍼랜드 인출을 위하여 직접 주소를 지정할 수 있는 기억장치 용량 결정

❖ 레지스터 번호 → 데이터 저장에 사용될 수 있는 레지스터의 개수 결정



2.4.2 명령어 형식

p106~p114

■ 오퍼랜드 필드 범위의 예

✓ 오퍼랜드1은 레지스터 번호를 지정하고, 오퍼랜드2는 기억장치 주소를 지정하는 경우

- 오퍼랜드1: 4 비트(2^4) → 16개의 레지스터 지정 가능
- 오퍼랜드2: 8 비트(2^8) → 0 ~ 255 번지의 기억장치 주소 지정 가능

✓ 두 개의 오퍼랜드들을 하나로 통합하여 사용하는 경우

- 오퍼랜드가 2의 보수로 표현되는 데이터라면,
  - 표현 범위: - 2048 ~ + 2047
- 오퍼랜드가 기억장치 주소라면,
  - 2^{12} = 4096 개의 기억장치 주소들 지정 가능



2.4.2 명령어 형식

■ 명령어 형식 (정리)

✓ **명령어 형식 = Op-code + Operand**

✓ 어떤 동작(명령)을 수행할 것인지?

➢ Op-code → add, load, branch, 등

✓ 데이터를 어디(소스)에서 찾을 것인지?

➢ Operand → CPU 레지스터, 메모리, I/O 위치

✓ 결과를 저장할 장소(목적지)?

➢ Operand → CPU 레지스터, 메모리, I/O 위치

✓ 다음 명령어의 위치?

➢ Operand

데이터 흐름 ←

add r0, r1, r3

add r0, r1, r3

add r0, r1, r3

br endloop

오퍼랜드 중 Source, Destination의 순서가 바뀌어 표기될 수도 있다
이 표기방법은 프로세서마다 다르니 주의할 필요가 있다

2.4.2 명령어 형식

p106~p114

■ 오퍼랜드의 수에 따른 명령어 분류

✓ 1-주소 명령어(one-address instruction)

- 오퍼랜드를 한 개만 포함하는 명령어 → 다른 한 오퍼랜드는 묵시적으로 AC가 됨
- [예] ADD X ; AC ← AC + M[X]

✓ 2-주소 명령어(two-address instruction)

- 두 개의 오퍼랜드를 포함하는 명령어
- [예] ADD R1, R2 ; R1 ← R1 + R2

  MOV R1, R2 ; R1 ← R2

  ADD R1, X ; R1 ← R1 + M[X]

✓ 3-주소 명령어(three-address instruction)

- 세 개의 오퍼랜드들을 포함하는 명령어
- [예] ADD R1, R2, R3 ; R1 ← R2 + R3

0-주소 명령어도 있음(오퍼랜드가 없는 명령어)

CPU의 발전에 따라 1-주소 명령어에서 3-주소 명령어로 바뀐다고 생각하면 되나, 근래의 CPU가 사용하는 상위 방식(3-주소 명령어) 내에서도 필요에 따라 하위 명령어 방식(1, 2-주소 명령어)도 사용함 → **사용하지 않는 오퍼랜드 필드 부분은 빈칸**으로 채워서 사용 (∵ 고정길이 명령어)

2.4.2 명령어 형식

p106~p114

■ 1-주소 명령어의 예

✓ 문제

➢ 길이가 16 비트인 1-주소 명령어에서 연산 코드가 5 비트인 경우의 명령어 형식을 정의하고, 주소지정 가능한 기억장치 용량을 결정하라

✓ 결과

➢ 주소지정 가능한 기억장치 용량 : 2^{11} = 2048 바이트

➢ 명령어 형식(instruction format)



2.4.2 명령어 형식

p106~p114

■ 2-주소 명령어의 예

✓ 문제

➢ 2-주소 명령어 형식을 사용하는 16-비트 CPU에서 연산 코드가 5 비트이고, 레지스터의 수는 8 개이다. (a) 두 오퍼랜드들이 모두 레지스터 번호인 경우와, (b) 한 오퍼랜드는 기억장치 주소인 경우의 명령어 형식을 정의하라

✓ 결과



(a) 두 개의 레지스터 오퍼랜드들을 가지는 경우



(b) 한 오퍼랜드는 기억장치 주소인 경우

2.4.2 명령어 형식

p106~p114

■ 3-주소 명령어의 예

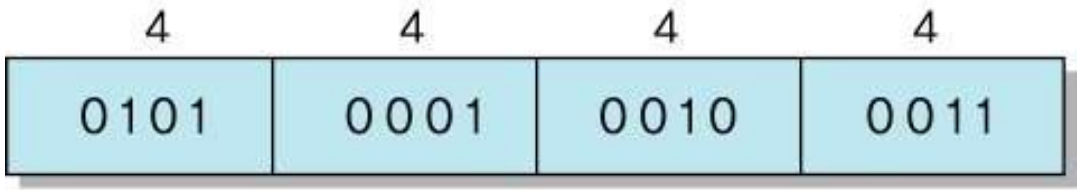
✓ 문제

➢ 3-주소 명령어 형식을 사용하는 16-비트 CPU에서 연산 코드가 4 비트이고, 레지스터의 수는 16 개이다. (a) 세 오퍼랜드들이 모두 레지스터 번호인 경우의 명령어 형식을 정의하라. (b) 이때 연산코드(ADD)는 0101b이며 오퍼랜드에 들어가는 레지스터의 이름은 각각 R1, R2, R3일 때의 명령어 형식을 정의하라. (ADD R1, R2, R3 명령어)

✓ 결과



(a) 명령어 형식



(b) ADD R1,R2,R3 명령어의 비트 배열

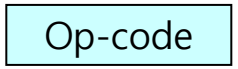
2.4.2 명령어 형식

이 내용은 명령어 형식의 발전 순서임

■ 오퍼랜드의 수에 따른 명령어 분류 (정리)

명령 순서	연산 동작
PUSH #a	SP ← SP + 1, [SP] ← #a
PUSH #b	SP ← SP + 1, [SP] ← #b
ADD	AC ← [SP], SP ← SP − 1 AC ← AC + [SP] [SP] ← AC

✓ 0-주소 명령어 : **오퍼랜드가 없는 명령어**

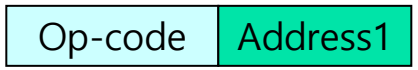


➢ 명령어 자체에 함축된 동작을 이용하여 연산에 사용할 데이터를 찾는다.

❖ 예) ADD

교재에는 없음

✓ 1-주소 명령어 : **오퍼랜드를 한 개만 포함하는 명령어**



➢ 연산에 필요한 피연산자 중 하나와 결과 값의 저장 장소로 누산기(AC)가 사용됨

❖ 예) ADD r1 ; AC ← r1 + AC

✓ 2-주소 명령어 : **두 개의 오퍼랜드를 포함하는 명령어**

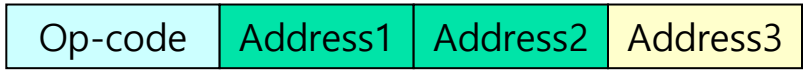


➢ 두 오퍼랜드 중 하나가 결과 값을 저장하는 장소로 사용됨.

➢ 즉 두 오퍼랜드 중 하나는 결과 값으로 덮어 써진다.

❖ 예) ADD r1, r2 ; r1 ← r1 + r2

✓ 3-주소 명령어 : **3개의 오퍼랜드를 포함하는 명령어**



➢ 연산에 필요한 두 개의 오퍼랜드, 연산 결과를 저장하는 한 개의 장소를 모두 갖고 있는 명령어

➢ 프로그램을 가장 짧게 작성 할 수 있으나, 명령어의 길이가 너무 길어지는 단점이 있다

❖ 예) ADD r1,r2,r3 ; r1 ← r2 + r3

2.4.2 명령어 형식

■ 오퍼랜드의 수에 따른 명령어 분류 (정리) - 계속

✓ 각 명령어 형식과 T = A + B 의 계산 예

구분	형식	예
0-주소	묵시적 주소 지정방식(스택 명령어 등)을 사용하여 연산 처리 비록 push, pop 명령어에서는 1-주소 방식을 사용하지만 **핵심 명령인 ADD는 0주소라서 0-주소 명령이라고 부름** → Stack Machine	PUSH A PUSH B **ADD** POP T
1-주소	모든 데이터 처리가 내장되어 있는 누산기(AC)에 의해 이루어짐. 주소 필드는 하나의 주소(레지스터, 메모리)를 지정 → Accumulator(누산기) Machine	LOAD A ADD B STORE T
2-주소	각 주소 필드는 2개의 주소(레지스터, 메모리)를 지정 → 범용 레지스터 Machine	MOV R1, A ADD R1, B MOV T, R1
3-주소	각 주소 필드가 3개의 주소(레지스터, 메모리)를 지정 → 범용 레지스터 Machine	ADD T, A, B

2.4.2 명령어 형식

p106~p114

■ 명령어 형식이 프로그래밍에 미치는 영향 (예)

✓ 예제: X = (A + B) x (C - D) 계산을 위한 어셈블리 프로그램 작성

➢ 아래와 같은 니모닉스를 가진 명령어들을 사용

- ❖ ADD : 덧셈
- ❖ SUB : 뺄셈
- ❖ MUL : 곱셈
- ❖ DIV : 나눗셈
- ❖ MOV : 데이터 이동
- ❖ LOAD : 기억장치로부터 데이터 적재
- ❖ STOR : 기억장치로 데이터 저장

2.4.2 명령어 형식

p106~p114

■ 명령어 형식이 프로그래밍에 미치는 영향 (예) - 계속

✓ 1-주소 명령어를 사용한 프로그램

계산식: X = (A + B) x (C - D)

```
LOAD   A      ;  AC ← M[A]
ADD    B      ;  AC ← AC + M[B]
STOR   T      ;  M[T] ← AC
LOAD   C      ;  AC ← M[C]
SUB    D      ;  AC ← AC - M[D]
MUL    T      ;  AC ← AC × M[T]
STOR   X      ;  M[X] ← AC
```

❖ 단, M[A]는 기억장치 A번지의 내용, T는 기억장치 내 임시 저장장소의 주소

➢ 프로그램의 길이 = 7

2.4.2 명령어 형식

p106~p114

■ 명령어 형식이 프로그래밍에 미치는 영향 (예) - 계속

✓ 2주소 명령어를 사용한 프로그램

계산식: X = (A + B) x (C - D)

```
MOV     R1, A       ; R1 ← M[A]
ADD     R1, B       ; R1 ← R1 + M[B]
MOV     R2, C       ; R2 ← M[C]
SUB     R2, D       ; R2 ← R2 - M[D]
MUL     R1, R2      ; R1 ← R1 ×  R2
MOV     X, R1       ; M[X] ← R1
```

➢ 프로그램의 길이 = 6

2.4.2 명령어 형식

p106~p114

■ 명령어 형식이 프로그래밍에 미치는 영향 (예) - 계속

✓ 3주소 명령어를 사용한 프로그램

계산식: X = (A + B) x (C - D)

```
ADD   R1, A, B        ;  R1 ← M[A] + M[B]
SUB   R2, C, D        ;  R2 ← M[C] - M[D]
MUL   X, R1, R2       ;  M[X] ← R1 ×  R2
```

- 프로그램의 길이 = 3
- 단점
  - 명령어의 길이가 증가한다
  - 명령어 해독 과정이 복잡해진다

결론

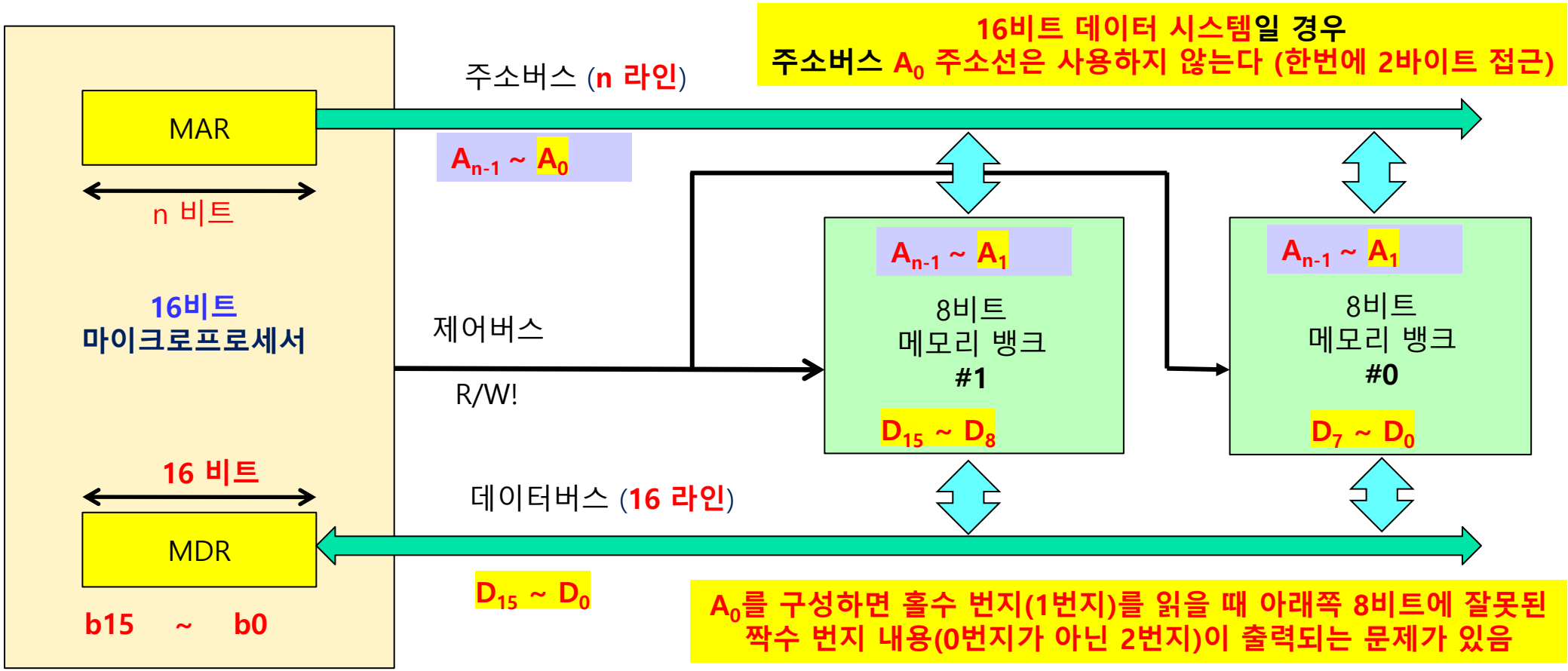
- 3-주소 명령어로 갈수록 **CPU 내부 구조는 복잡**해지나 스택(메모리 접근) 작업 및 명령어의 숫자가 줄어서 **성능이 대폭 향상됨**

2.4.3 주소지정 방식

■ 추가: CPU와 메모리 구성 (메모리 정렬이 필요한 이유)

✓ 16비트 CPU에서 16비트 MDR 구조를 사용하는 CPU (8비트 메모리 모듈을 복수 개를 사용하여 메모리 시스템을 구성하는 경우)

➢ 2바이트씩 접근하기 때문에 홀수 주소(A_0=1)를 접근하는 경우 2번의 메모리 접근이 필요(뒤쪽 학습)



2.4.3 주소지정 방식

■ 추가: 명령어와 메모리

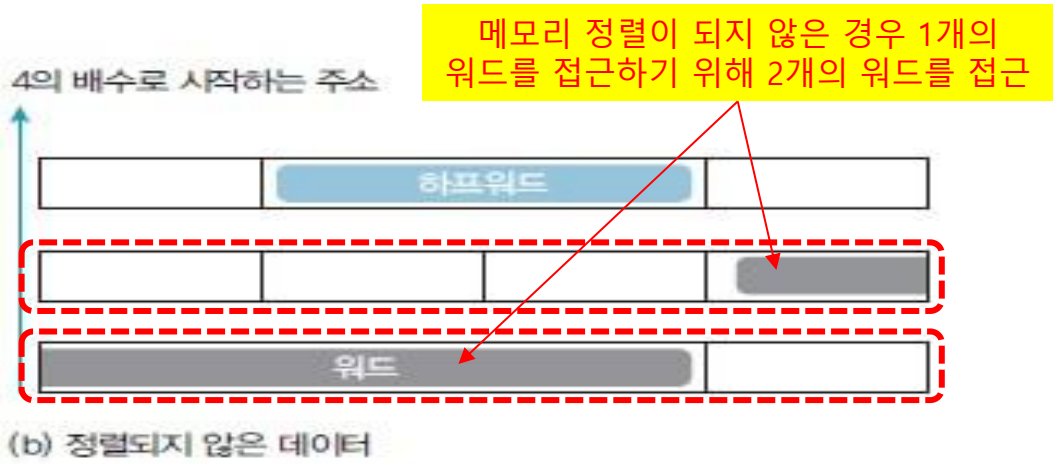
✓ 메모리 주소 지정 단위

- 대부분의 CPU는 **CPU 워드와 동일한 메모리 크기**로 메모리에 접근 → 32비트 CPU는 무조건 4바이트 단위로 주소를 생성(0, 4, 8,....)
- 그러나 하나의 메모리 주소에는 일반적으로 워드 크기가 아닌 **바이트 단위**로 지정하여 사용
  - ❖ 아주 특별한 경우만 워드 단위로 주소를 지정 (몇몇 MCU들에서 내부 프로그램 메모리를 접근하는 경우)

✓ 메모리 정렬(Align)

- 워드와 바이트의 크기 차이로 인해 메모리 정렬이 필요 → 어셈블리 언어에서는 align 명령 지원, 고급 언어는 컴파일러가 자동 정렬
  - ❖ 미정렬시 1개의 워드를 접근하기 위해 복수 번의 메모리 접근이 필요
- 1 워드가 4 바이트인 경우의 예





2.4.3 주소지정 방식

■ 추가: 명령어와 메모리

✓ 엔디안(Endian) 방식

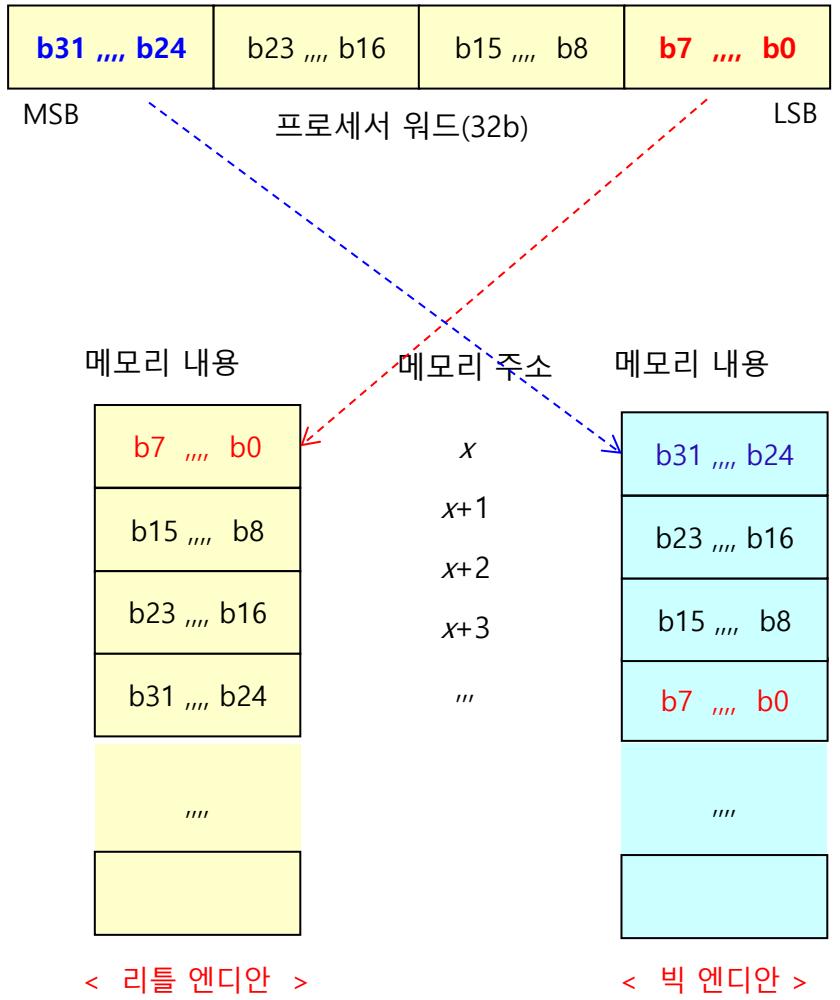
➢ CPU가 워드단위(복수 byte)로 메모리를 접근할 때 각 byte들을 어떤 순서로 메모리에 저장(또는 읽음)하는 지에 대한 규칙

1) 리틀-엔디안(Little-Endian)

- ❖ 낮은 메모리 주소에 낮은 Byte 부터 차례로 저장
- ❖ 인텔 80x86계열, DEC의 CPU 계열에서 주로 사용

2) 빅-엔디안(Big-Endian)

- ❖ 낮은 메모리 주소에 높은 Byte 부터 차례로 저장
- ❖ 주로 네트워크(통신)에서 사용
- ❖ TCP/IP 전송방식, IBM 360/370, 모토롤라 68x, Sparc, PA 계열에서 주로 사용



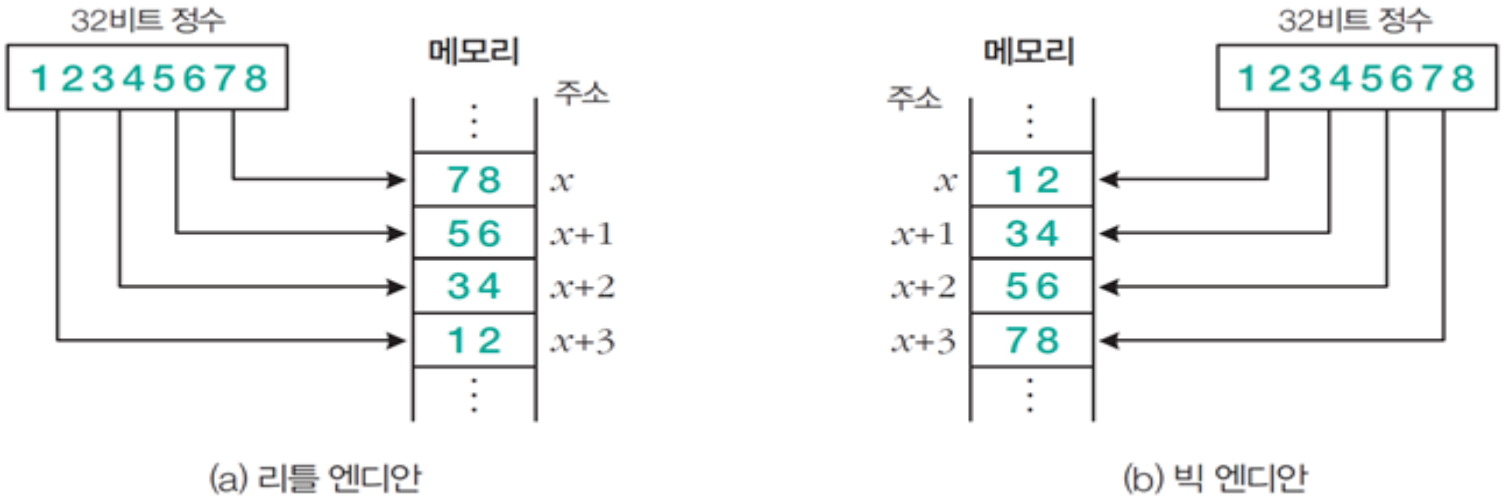
2.4.3 주소지정 방식

■ 추가: 명령어와 메모리

✓ 엔디안(Endian) 방식

3) 바이-엔디안(Bi-Endian)

- ❖ 엔디안을 선택할 수 있도록 설계된 방법으로 ARM, PowerPC, DEC Alpha, MIPS 등이 있다.
- ❖ 근래의 CPU들은 두가지 방식을 모두 지원(ARM 등)



(a) 리틀 엔디안 (b) 빅 엔디안

2.4.3 주소지정 방식

p114~p126

■ 주소지정 방식(addressing mode)

- ✓ 명령어 실행에 필요한 오퍼랜드의 주소를 결정하는 방식
- ✓ 다양한 주소지정 방식을 사용하는 이유
  - ➢ 제한된 수의 오퍼랜드 비트들을 이용하여, 사용자(프로그래머)가 여러 가지 방법으로 오퍼랜드의 주소를 결정하도록 해주며, 더 큰 용량의 기억장치를 사용할 수 있도록 하기 위함
- ✓ 명령어 내 오퍼랜드 필드의 종류 → 한 종류 또는 복수개가 혼용된다.
  - ➢ 기억장치 주소
    - ❖ 데이터가 저장된 기억장치의 위치를 지정
  - ➢ 레지스터 번호
    - ❖ 데이터가 저장된 레지스터를 지정
  - ➢ 데이터
    - ❖ 명령어의 오퍼랜드 필드에 데이터가 포함

주소지정 방식은 CPU마다 제공하는 종류가 다르고 또한 표기법도 차이가 많다

2.4.3 주소지정 방식

■ 주소지정 방식(addressing mode)의 필요성 예 - 1/2

✓ ARM CPU (32비트 계열)

➢ 32개의 시스템 버스 주소선(A_{31}~A_0) → 2^{32} = 4Giga 개의 주소

➢ 32비트 고정길이 명령어

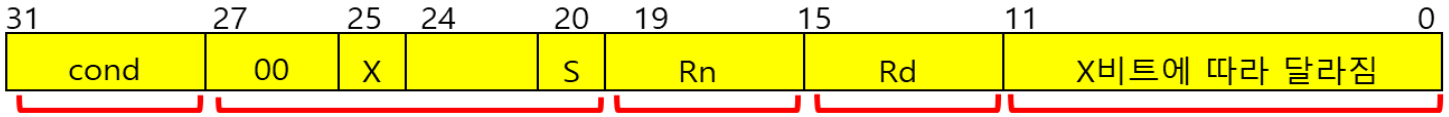
① Condition → 4비트
② Opcode(연산자) → 8비트
③ Source1 → 4비트
④ Destination → 4비트
⑤ Source2 → 12비트

오퍼랜드 부분 (기본 3주소 방식)

➢ 만약 1주소 방식을 사용해도 **명령어 내에서 주소를 지정할 수 있는 비트의 수는 총 20비트**(4 + 4 + 12)

❖ 실제로는 1주소 방식을 사용하지 않지만 주소지정 방식의 설명을 위해 사용한다고 가정함.

➢ 오퍼랜드 부분에 상수만을 사용해도 **20비트의 범위만 지정이 가능**



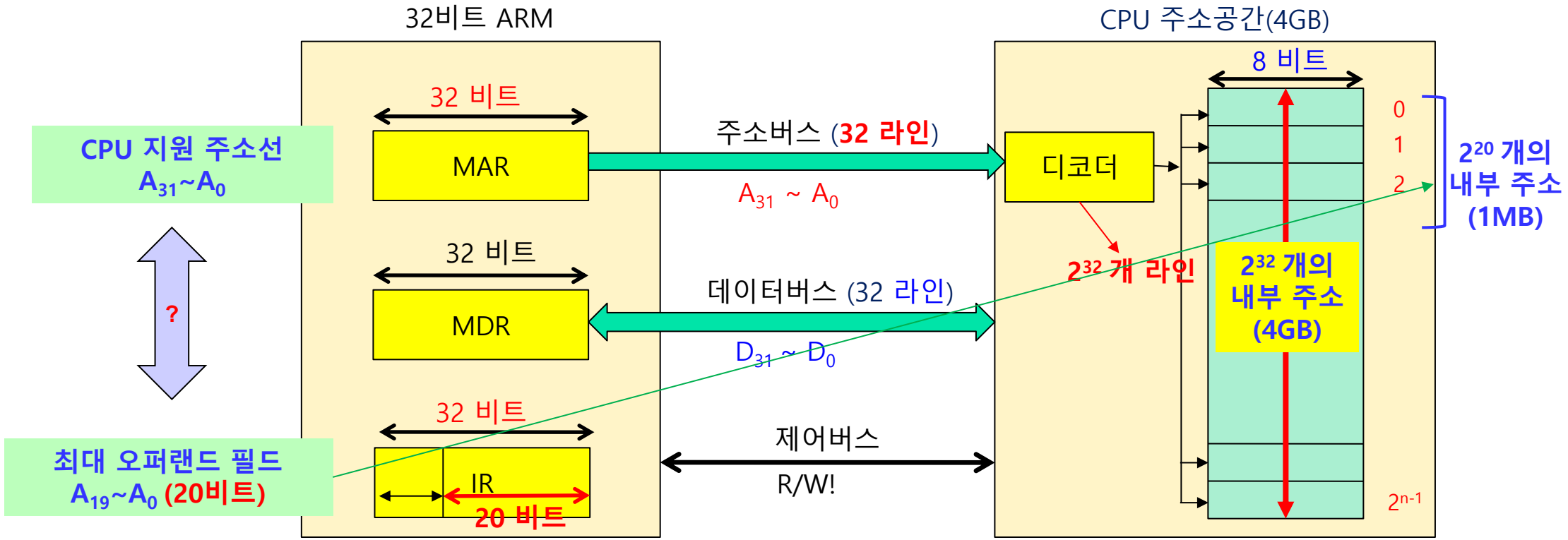
ADD Rd, Rn, Op2 // Rd = Rn + Op2
Rd = destination register
Rn = operand 1 (항상 register임)
Op2 = operand 2 (register 혹은 immediate 값)

2.4.3 주소지정 방식

■ 주소지정 방식(addressing mode)의 필요성 예 - 2/2

✓ ARM CPU (32비트 계열)

- ➢ 20비트를 하나의 오퍼랜드 필드로 모두 사용해도 **A_{19}~A_0 (1Mega) 즉 20개의 주소선만 지정 가능**
- ➢ 따라서 주소선 **32개(4Giga)를 모두 사용할 방법**이 필요 → **주소지정 방식의 개념이 필요**



2.4.3 주소지정 방식

p114~p126

■ 주소지정 방식에서 사용하는 기호

- ✓ *EA*
  - ➢ 유효 주소(Effective Address), 데이터가 저장된 기억장치의 실제 주소(시스템 버스의 주소 버스)
  - ➢ 레지스터인 경우는 내부 버스로 접근
- ✓ *A*
  - ➢ 명령어 내의 주소 필드 내용 (오퍼랜드 필드의 내용이 기억장치 주소인 경우)
- ✓ *R*
  - ➢ 명령어 내의 레지스터 번호 (오퍼랜드 필드의 내용이 레지스터 번호인 경우)
- ✓ *(A)*
  - ➢ 기억장치 A 번지 내부에 저장된 내용
  - ➢ *[A]*로 표기하기도 함
- ✓ *(R)*
  - ➢ 레지스터 R 내부에 저장된 내용
  - ➢ *[R]*로 표기하기도 함

2.4.3 주소지정 방식

p114~p126

■ 주소지정 방식의 종류

1) (메모리) 직접 주소지정 방식(direct addressing mode)
2) (메모리) 간접 주소지정 방식(indirect addressing mode)
3) 묵시적 주소지정 방식(implied addressing mode)
4) 즉시 주소지정 방식(immediate addressing mode)
5) 레지스터 (직접) 주소지정 방식(register addressing mode)
6) 레지스터 간접 주소지정 방식(register-indirect addressing mode)
7) 변위 주소지정 방식(displacement addressing mode)
   - ➢ 상대 주소지정 방식(relative addressing mode)
   - ➢ 인덱스 주소지정 방식(indexed addressing mode)
   - ➢ 베이스-레지스터 주소지정 방식(base-register addressing mode)

주소지정 방식은 CPU마다 제공하는 종류가 다르고 또한 표기법도 차이가 많다

2.4.3 주소지정 방식

p114~p126

1) (메모리) 직접 주소지정 방식(direct addressing mode)

✓ 오퍼랜드 필드의 내용이 유효 주소(*EA*)가 되는 방식

EA = A

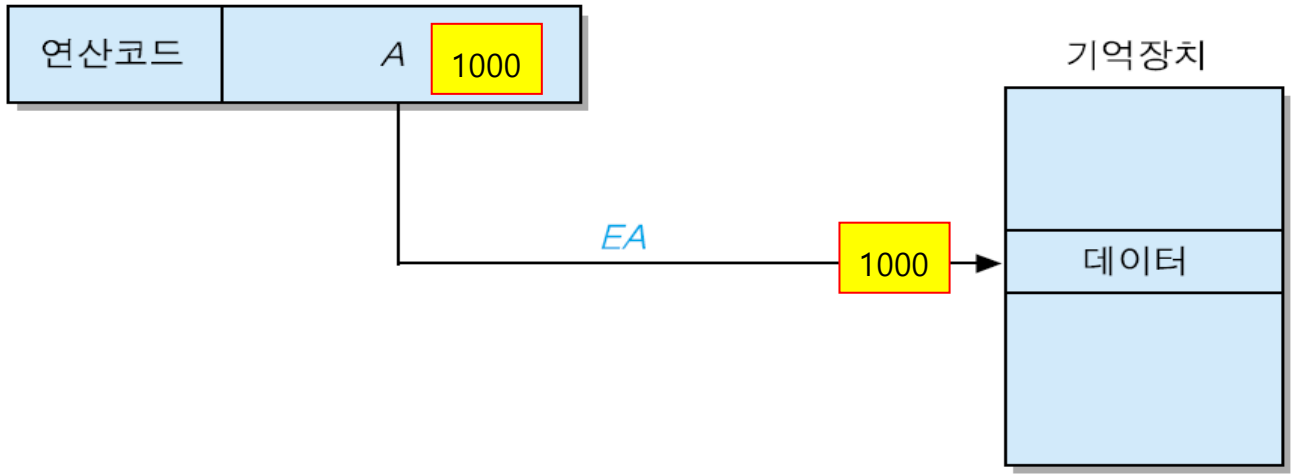
```
MOV   R1, 1000  ; R1 ← M[1000]
MOV   1000, R1  ; M[1000] ← R1
```

✓ 장점

➢ 데이터 인출을 위하여 한 번의 기억장치 액세스만 필요

✓ 단점

➢ 연산 코드를 제외하고 남은 비트들만 주소 비트로 사용될 수 있기 때문에 직접 지정할 수 있는 기억장소의 수가 제한



2.4.3 주소지정 방식

p114~p126

1) (메모리) 직접 주소지정 방식의 예제

모든 주소지정방식의 예제들은 하나의 주소선에 한 워드씩 지정되는 형태임

예제 2–6

CPU 내부 레지스터들과 주기억장치에 그림 2–24와 같은 값들이 저장되어 있다고 가정하자. 여기서, CPU 레지스터 및 각 기억 장소의 폭(width)은 16비트이며, 그림에서 모든 값들은 편의상 10진수로 표시하였다(그림 2–24는 [예제 2–7] 및 [예제 2–9]에서도 공통적으로 사용됨).

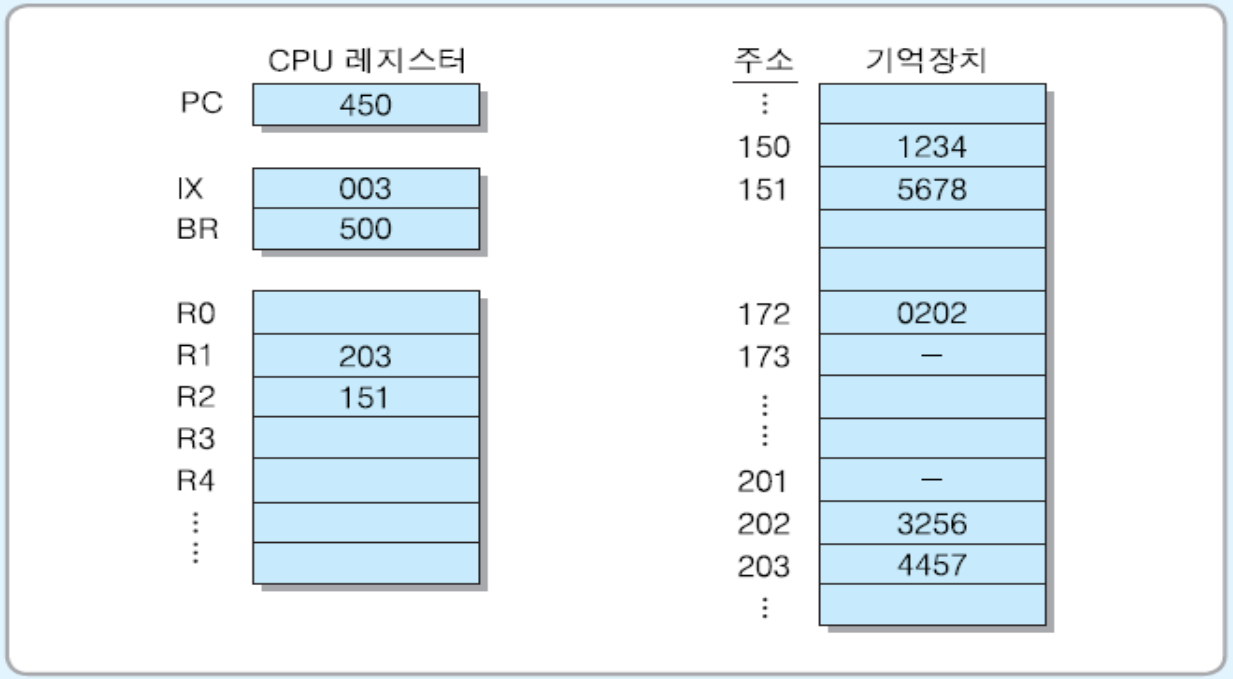


그림 2–24 예제 2–6, 2–7, 2–9를 위한 그림

(1) 직접 주소지정 방식을 사용하는 명령어의 주소 필드(A)에 저장된 내용이 150일 때, 유효 주소(*EA*) 및 그에 의해 인출되는 데이터를 구하라.

(2) 명령어 길이가 16비트이고 연산 코드가 5비트라면, 이 명령어에 의해 직접 주소 지정 될 수 있는 기억장치 용량은 얼마인가?

풀이

(1) EA = 150이므로, 기억장치 150번지에 저장된 데이터 '1234'가 인출된다.

(2) 주소 필드가 11비트이므로, 직접 주소지정 할 수 있는 기억장치 용량은 2^{11} = 2048 단어가 된다. 그런데 각 기억 장소에 저장되는 데이터의 비트 수가 16비트(2바이트)이므로, 기억장치 용량은 4096바이트로 표현할 수도 있다.

2.4.3 주소지정 방식

p114~p126

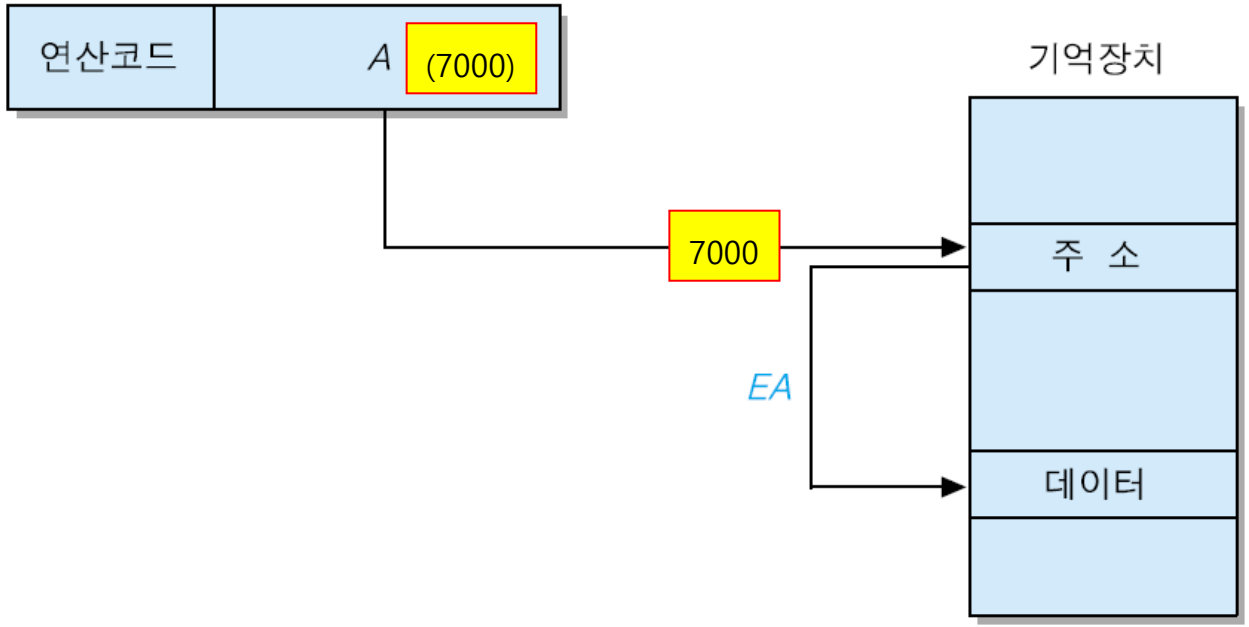
2) (메모리) 간접 주소지정 방식(indirect addressing mode)

✓ 오퍼랜드 필드에 기억장치 주소가 저장되어 있지만, 그 주소가 가리키는 기억 장소에 데이터의 유효 주소를 저장해두는 방식

EA = (A)

```
ADD R1, (7000)    ; R1 ← R1 + M[M[7000]]
ADD R1, @(7000) ; R1 ← R1 + M[M[7000]]
```

※ @ 생략 가능



2.4.3 주소지정 방식

p114~p126

2) (메모리) 간접 주소지정 방식 - 계속

✓ 장점

- 최대 기억장치용량이 워드의 길이에 의하여 결정 → 주소지정 가능한 기억장치 용량 확장
- 워드 길이가 n 비트라면, 최대 2^n 개의 기억 장소에 대한 주소지정이 가능

✓ 단점

- 실행 사이클 동안에 두 번의 기억장치 액세스가 필요
  - ① 첫 번째 액세스 → 주소 인출
  - ② 두 번째 액세스 → 그 주소가 지정하는 기억 장소로부터 실제 데이터 인출

✓ 명령어 형식에 간접비트(I) 필요

- 만약 I = 0 → 직접 주소지정 방식
- 만약 I = 1 → 간접 주소지정 방식 → 간접 사이클 실행

✓ 다단계(multi-level) 간접 주소지정 방식

- EA = ((. . (A) . .))



긴접비트를 사용하지 않고 연산자(op-code) 자체를 구분하여 구현하는 CPU도 있음

p114~p126

2) (메모리) 간접 주소지정 방식의 예제

예제 2-6

CPU 내부 레지스터들과 주기억장치에 그림 2-24와 같은 값들이 저장되어 있다고 가정하자. 여기서, CPU 레지스터 및 각 기억 장소의 폭(width)은 16비트이며, 그림에서 모든 값들은 편의상 10진수로 표시하였다(그림 2-24는 [예제 2-7] 및 [예제 2-9]에서도 공통적으로 사용됨).

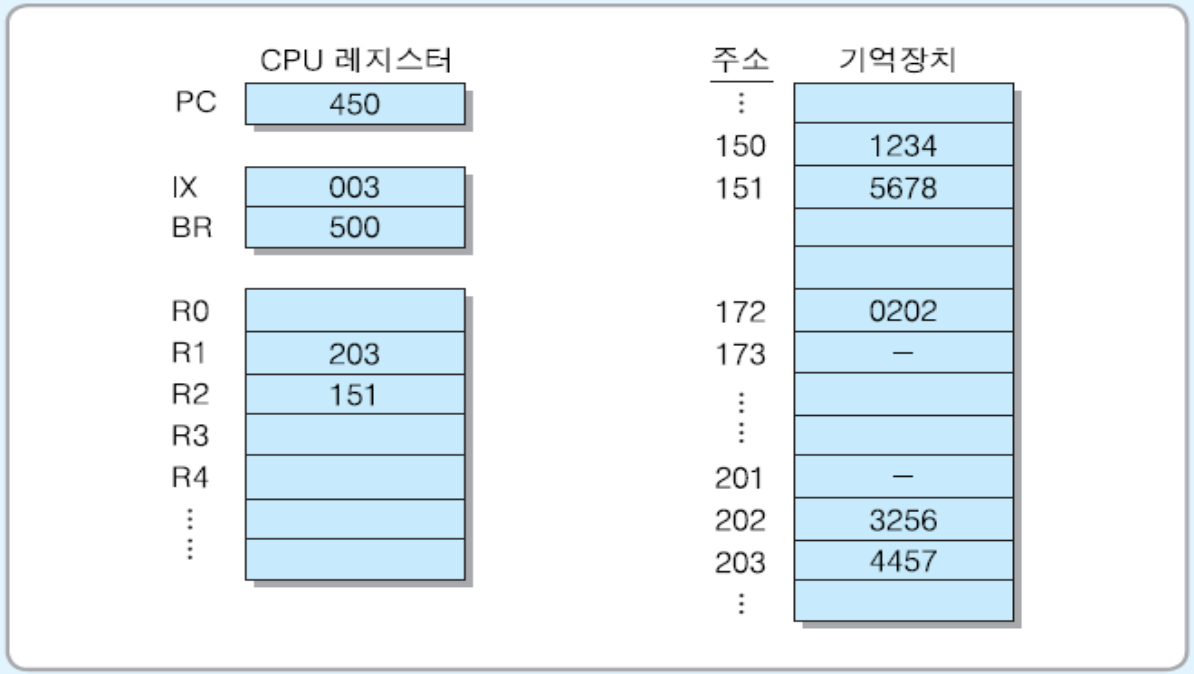


그림 2-24 예제 2-6, 2-7, 2-9를 위한 그림

예제 2-7

CPU 레지스터들과 주기억장치에 그림 2-24와 같은 값들이 저장되어 있을 때 아래 물음에 답하라.

(1) 간접 주소지정 방식을 사용하는 명령어의 주소 필드(*A*)에 저장된 내용이 '172'라고 가정했을 때, 유효 주소(*EA*) 및 그에 의해 인출되는 데이터를 구하라.

(2) 이 명령어에 의해 주소지정 될 수 있는 기억장치 용량은 얼마인가?

풀이

(1) *EA*는 그림 2-24의 기억장치 172번지에 저장되어 있는 '202'이다. 따라서 명령어 실행에 사용될 데이터로는 기억장치 202번지에 저장되어 있는 '3256'이 인출된다.

(2) 주소(*EA*)의 길이는 16비트가 되므로, 주소지정 할 수 있는 기억장치 용량은 2^{16} = 64Kword(128KByte)가 된다.

2.4.3 주소지정 방식

p114~p126

3) 묵시적 주소지정 방식(implied addressing mode)

- ✓ 명령어 실행에 필요한 데이터의 위치가 묵시적으로 지정되는 방식
- ✓ [예]
  - ➢ 'CLI' 명령어 → 상태레지스터 내부의 인터럽트 비트를 해제해 CPU가 인터럽트 요청 여부를 조사하지 않는다
  - ➢ 'SHL' 명령어 → 누산기의 내용을 좌측으로 시프트(shift)
  - ➢ 'PUSH R1' 명령어 → 레지스터 R1의 내용을 스택에 저장
    - ❖ SP가 가리키는 기억장소에 R1의 내용을 저장한다는 것이 묵시적으로 정해져 있음
- ✓ 장점
  - ➢ 명령어 길이가 짧다
- ✓ 단점
  - ➢ 종류가 제한된다

2.4.3 주소지정 방식

p114~p126

4) 즉시 주소지정 방식(immediate addressing mode)

- ✓ 데이터가 명령어에 포함되어 있는 방식 (오퍼랜드 필드의 내용이 연산에 사용할 실제 데이터)
- ✓ 즉치 주소방식 이라고도 부름
- ✓ 용도
  - ➢ 프로그램에서 레지스터나 변수의 초기 값을 어떤 상수 값(constant value)으로 설정하는 데 사용
- ✓ 장점
  - ➢ 데이터를 인출하기 위하여 기억장치를 액세스할 필요가 없음
- ✓ 단점
  - ➢ 상수 값의 크기가 오퍼랜드 필드의 비트 수에 의해 제한됨




```
MOV   R1,#200    ; R1 ← 200
LOAD  #200       ; AC ← 200
```

2.4.3 주소지정 방식

p114~p126

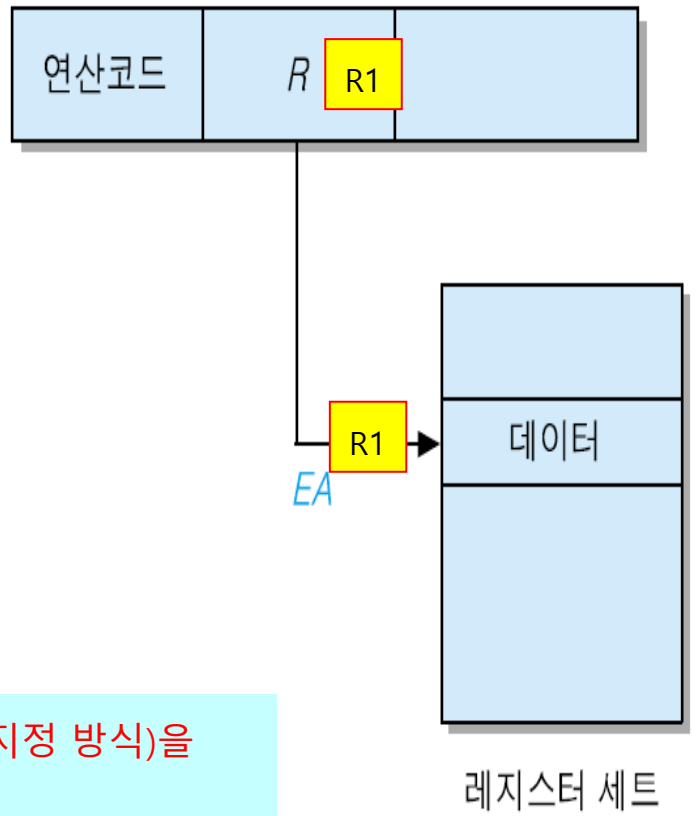
5) 레지스터 직접 주소지정 방식

- ✓ 연산에 사용될 데이터가 레지스터에 저장되어 있고 명령어의 오퍼랜드가 해당 레지스터를 가리키는 방식

  EA = R

- ✓ 주소지정에 사용될 수 있는 레지스터들의 수 = 2^k 개
  - ➢ 단, k는 R 필드의 비트 수
- ✓ 장점
  - ➢ 오퍼랜드 필드의 비트 수가 적어도 된다
  - ➢ 데이터 인출을 위하여 기억장치 액세스가 필요 없다
- ✓ 단점
  - ➢ 데이터가 저장될 수 있는 공간이 CPU 내부 레지스터들로 제한

ADD R1, R2 ; R1 ← R1+ R2



RISC CPU들은 속도가 느린 메모리를 이용한 데이터의 입출력(주소지정 방식)을 배제하기 위해 많은 종류의 레지스터들을 가지고 있다 → 속도 개선

2.4.3 주소지정 방식

p114~p126

6) 레지스터 간접 주소지정 방식

✓ 오퍼랜드 필드(레지스터 번호)가 가리키는 레지스터의 내용을 유효 주소로 사용하여 실제 데이터를 인출하는 방식

EA = (R)

✓ 장점

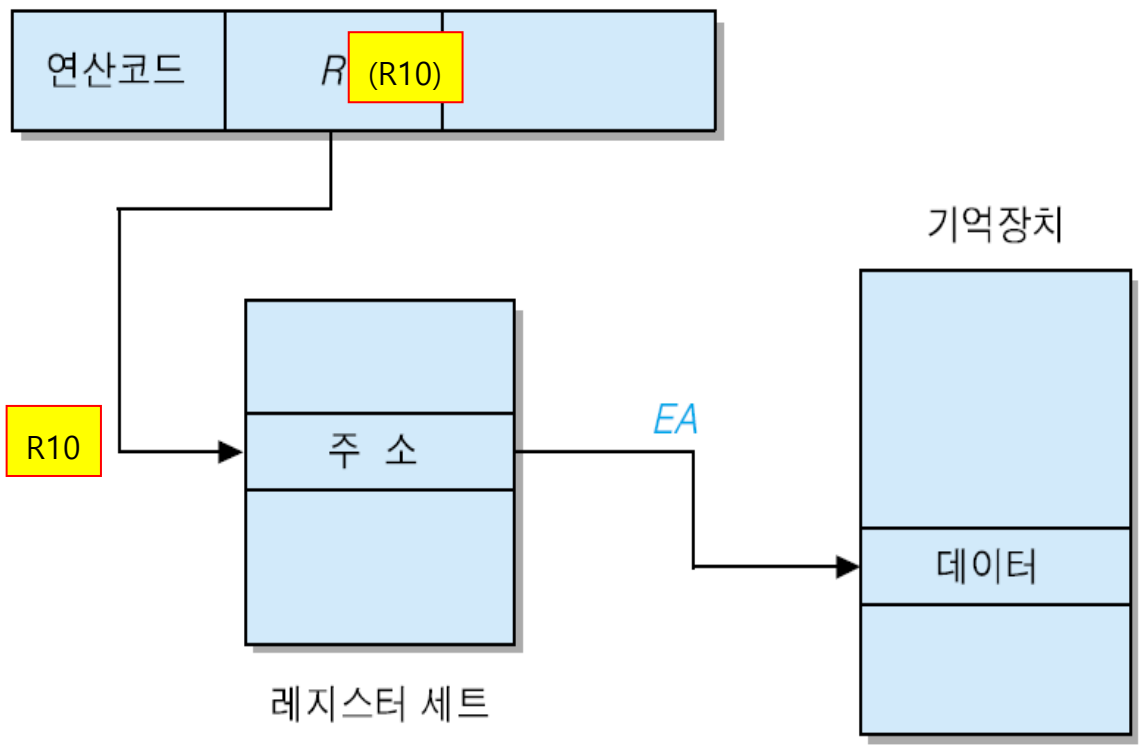
➢ 레지스터의 길이 = 16 비트라면

❖ 주소지정 영역은 2^{16} = 64K

➢ 레지스터의 길이 = 32 비트라면

❖ 주소지정 영역: 2^{32} = 4G

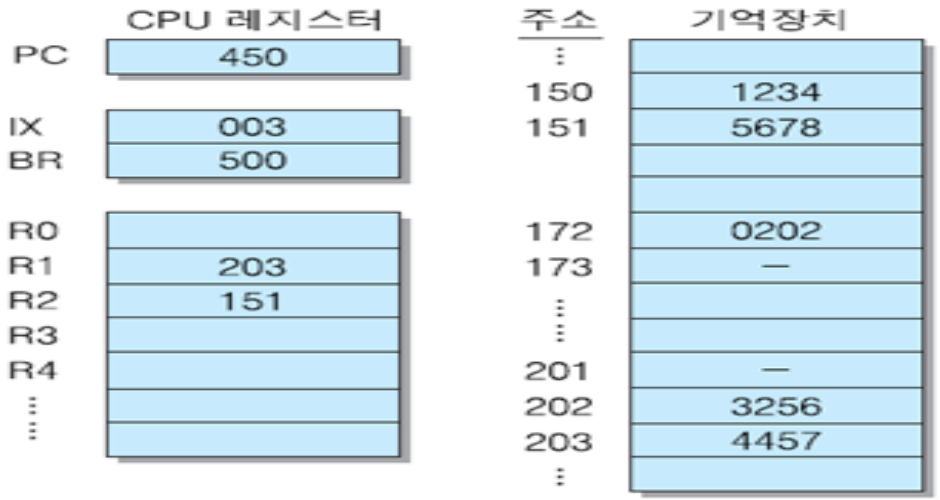
LOAD R1, (R10) ; R1 ← M[R10]

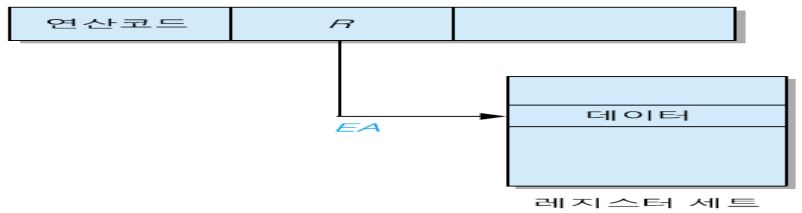


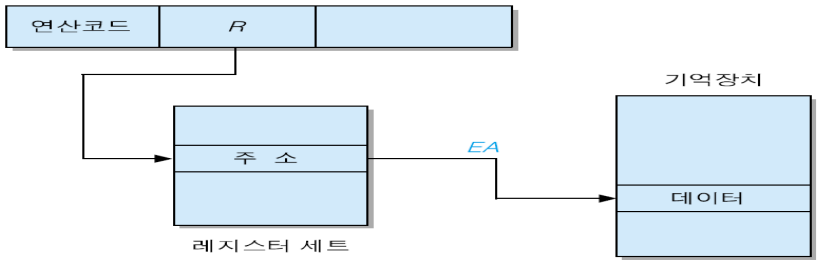
2.4.3 주소지정 방식

p114~p126

6) 레지스터 간접 주소지정 방식 예제







예제 2-9

CPU 내부 레지스터들과 주기억장치에 그림 2-24와 같은 값들이 저장되어 있고, 그림 2-28과 2-29의 명령어 형식에서 레지스터 필드 R에는 '2'가 저장되어 있다고 가정하자.

(1) 레지스터 주소지정 방식이 사용된다면, 연산 처리 과정에서 어떤 데이터가 사용될 것인가?

(2) 레지스터 간접 주소지정 방식이 사용된다면, 어떤 데이터가 사용될 것인가?

풀이

(1) R2에 저장되어 있는 데이터 '151'이 사용된다.

(2) 기억장치 151번지에 저장되어 있는 데이터 '5678'이 사용된다.

2.4.3 주소지정 방식

p114~p126

7) 변위 주소지정 방식(displacement addressing)

✓ 직접 주소지정과 레지스터 간접 주소지정 방식의 조합

EA = A + (R)

✓ 사용되는 레지스터에 따라 여러 종류의 변위 주소지정 방식 가능

① PC

❖ 상대 주소지정 방식(relative addressing mode)

② 인덱스 레지스터

❖ 인덱스 주소지정 방식(indexed addressing mode)

③ 베이스 레지스터

❖ 베이스-레지스터 주소지정 방식(base-register addressing mode)

2.4.3 주소지정 방식

p114~p126

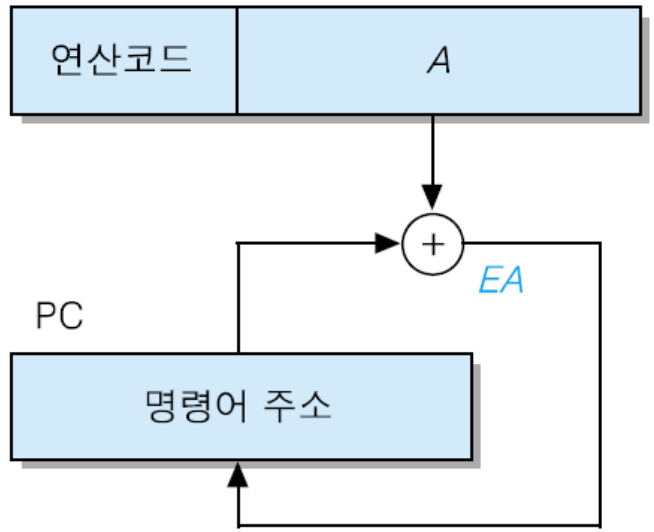
7-1) 상대 주소지정 방식(relative addressing mode)

- ✓ 프로그램 카운터(PC)를 레지스터로 사용하여 EA를 계산

  EA = A + (PC) (단, A는 2의 보수)

- ✓ 주로 분기 명령어에서 사용
  - ➢ $A > 0$ → 앞(forward) 방향으로 분기
  - ➢ $A < 0$ → 뒷(backward) 방향으로 분기
- ✓ 장점
  - ➢ 전체 기억장치 주소가 명령어에 포함되어야 하는 일반적인 분기 명령어보다 적은 수의 비트만 필요
- ✓ 단점
  - ➢ 분기 범위가 오퍼랜드 필드의 길이에 의해 제한
    - ❖ 오퍼랜드 비트들로 표현 가능한 2의 보수 범위

LOAD R1, ***X(PC)*** ; *X* = 변위주소 → 유효주소 = *X* + PC



2.4.3 주소지정 방식

p114~p126

7-1) 상대 주소지정 방식

예제 2–10

상대 주소지정 방식을 사용하는 JUMP 명령어가 450번지에 저장되어 있다.

(1) 만약 오퍼랜드 A = '21'이라면, 몇 번지로 점프하는가?

(2) 만약 오퍼랜드 A = '–50'이라면, 몇 번지로 점프하는가?

풀이

(1) 이 명령어가 인출된 후에는 PC의 내용이 451로 증가된다. 따라서 451 + 21 = 472번지로 점프하게 된다.

(2) (1)번과 같은 원리로, 451 − 50 = 401번지로 점프하게 된다.

p114~p126

7-2) 인덱스 주소지정 방식(indexed addressing mode)

✓ 인덱스 레지스터의 내용과 변위 A를 더하여 유효 주소를 결정

EA = (IX) + A

LOAD R1, ***X***(R_{index}) ; R_{index} = 인덱스 레지스터 → 유효주소= X + R_{index}

✓ 인덱스 레지스터(IX) → 인덱스(index) 값을 저장하는 특수-목적 레지스터

✓ 주요 용도

➢ 배열 데이터 액세스

✓ 자동 인덱싱(auto-indexing)

➢ 명령어가 실행될 때마다 인덱스 레지스터의 내용이 자동적으로 증가 혹은 감소

❖ 상태 레지스터에 방향(증가/감소)을 결정하는 플래그가 있다 → D(direction) flag

➢ 이 방식이 사용된 명령어가 실행되면 아래의 두 연산이 연속적으로 수행됨

EA = (IX) + A

IX ← IX + 1 또는 IX - 1

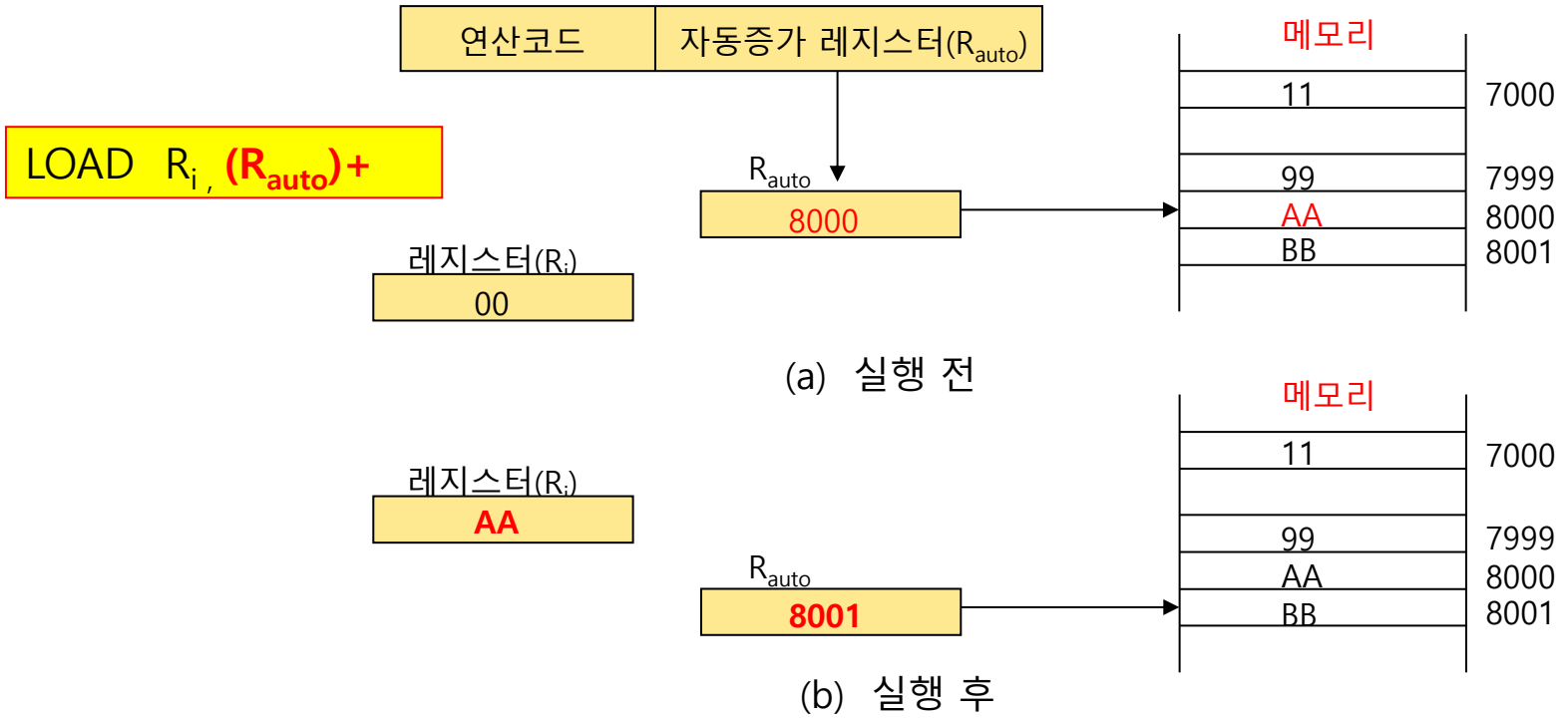
LOAD R_i, **(R_{auto})+**
LOAD R_i, **+(R_{auto})**
LOAD R_i, **(R_{auto})-**
LOAD R_i, **-(R_{auto})**

p114~p126

7-2) 인덱스 주소지정 방식(indexed addressing mode)

✓ 자동증가 주소지정 방식

- 자동증가 레지스터를 사용하며, 이 레지스터의 출력이 유효주소가 된다
- 메모리로 접근한 후, 자동증가 레지스터의 값이 자동으로 증가



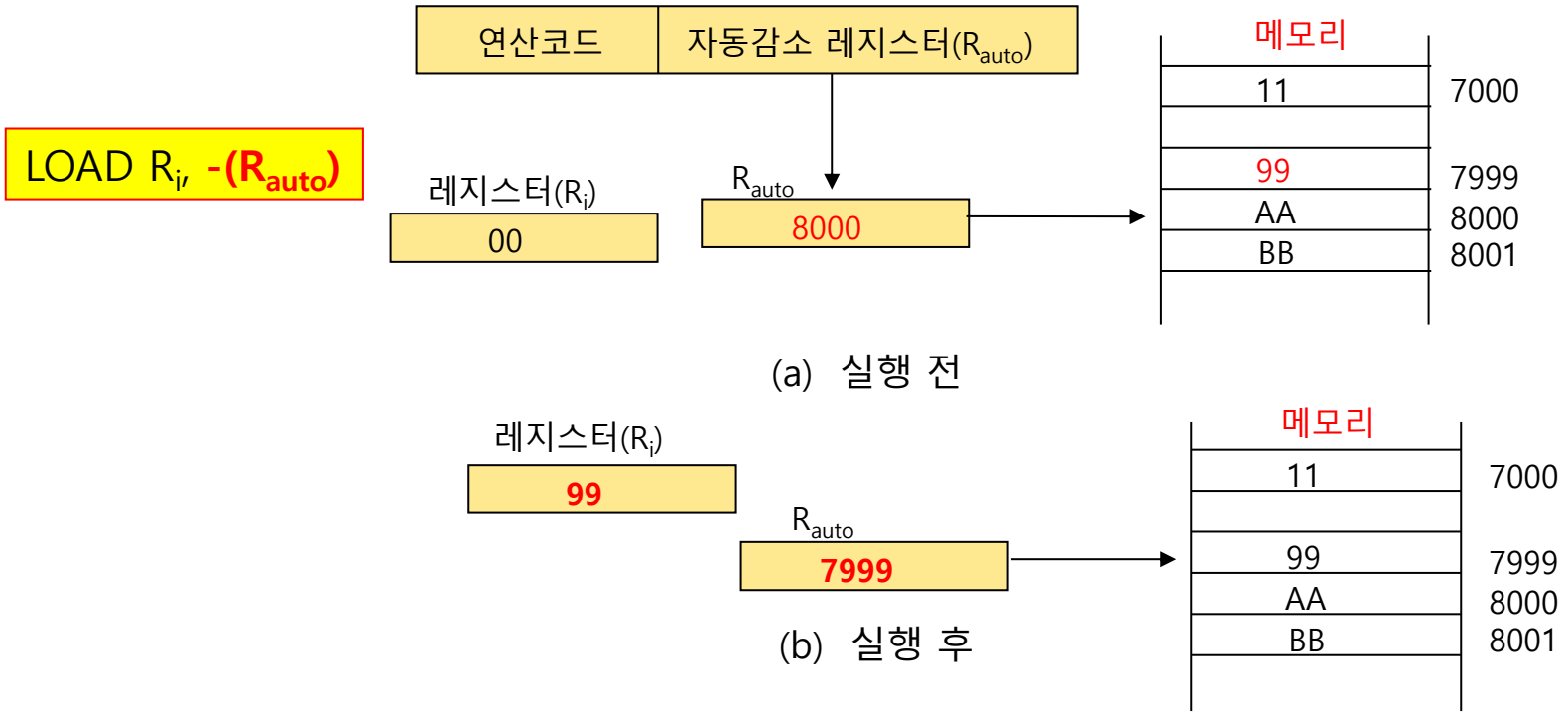
2.4.3 주소지정 방식

p114~p126

7-2) 인덱스 주소지정 방식(indexed addressing mode)

✓ 자동감소 주소지정 방식

- 자동감소 레지스터를 사용하며, 이 레지스터의 출력이 유효주소가 된다
- 자동감소 레지스터의 값이 자동으로 감소한 뒤에 메모리로 접근한다

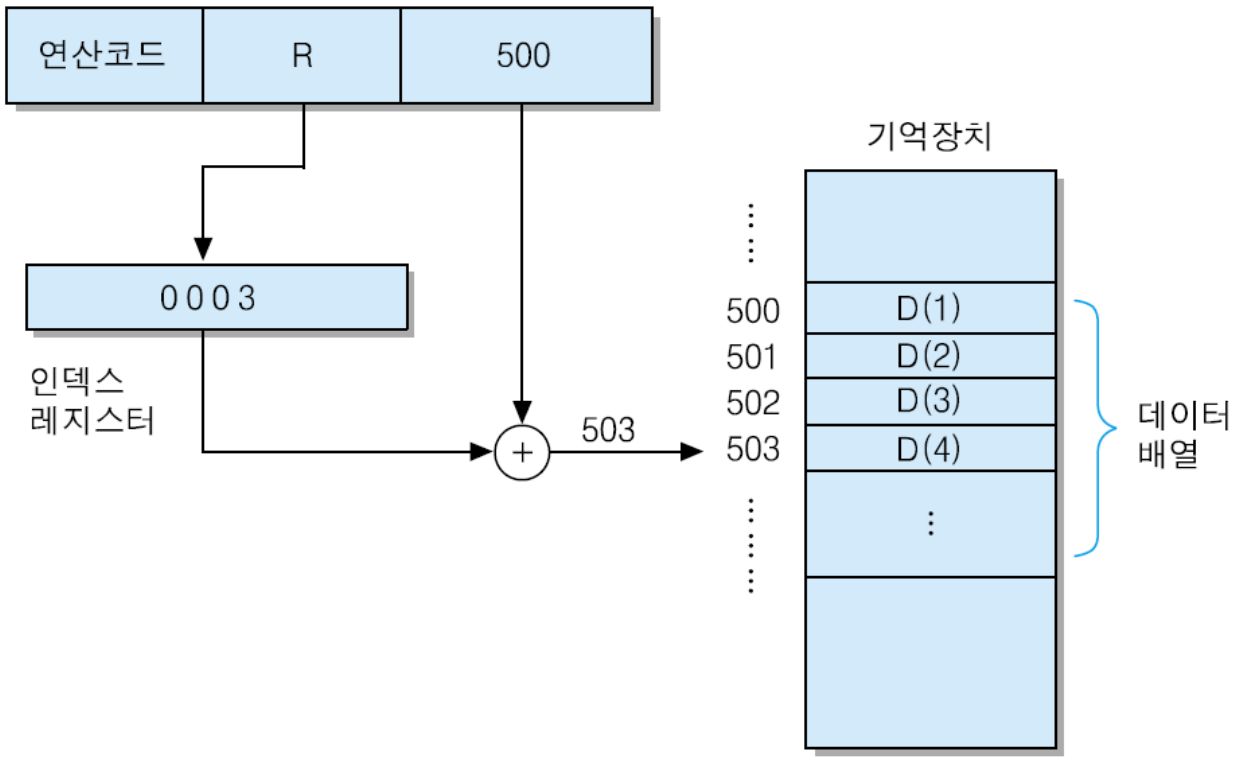


2.4.3 주소지정 방식

p114~p126

7-2) 인덱스 주소지정 방식의 예제

- ✓ 데이터 배열이 기억장치의 500 번지부터 저장되어 있고
- ✓ 명령어의 주소 필드에 '500' 이 포함되어 있을 때
- ✓ 인덱스 레지스터의 내용 (IX) = 3 이라면 → 데이터 배열의 네 번째 데이터 액세스



2.4.3 주소지정 방식

p114~p126

7-3) 베이스-레지스터 주소지정 방식

✓ 베이스 레지스터의 내용과 변위 A를 더하여 유효 주소를 결정

$$EA = (BR) + A$$

✓ 주요 용도

- 프로그램의 위치 지정(혹은 변경) 에 사용

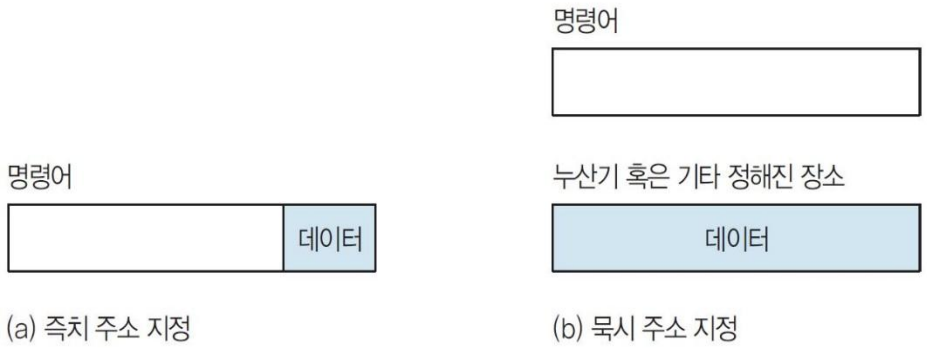
✓ 사용 예

- 다중프로그래밍(multiprogramming) 환경에서 프로그램 코드 및 데이터를 다른 위치로 이동시켜야 할 때, 분기 명령어나 데이터 액세스 명령어들의 주소 필드 내용을 바꿀 필요 없이, BR 레지스터의 내용만 변경하면 된다.
- 운영체제에서 프로세스(또는 쓰레드)의 변환시에 많이 사용

2.4.3 주소지정 방식

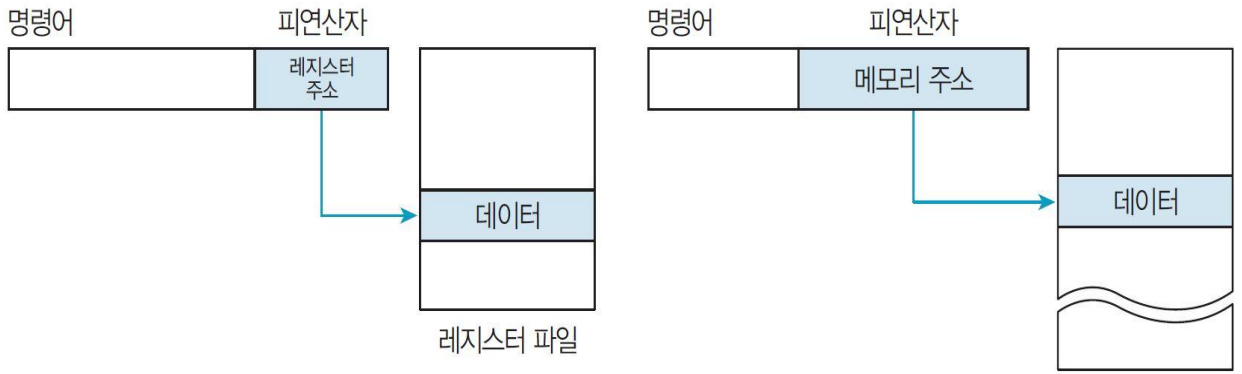
■ 단계별 분류로 정리한 주소 지정 방식

1) 0-단계 주소 지정



(a) 즉치 주소 지정 (b) 묵시 주소 지정

2) 1-단계 주소 지정

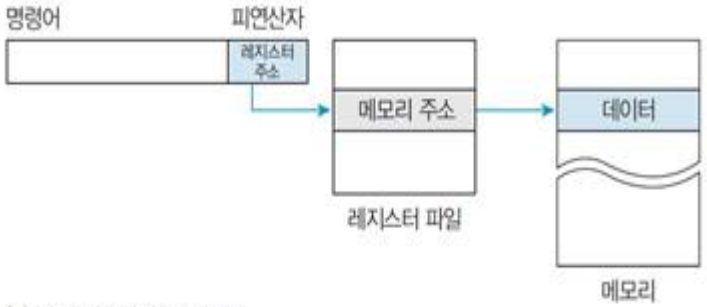


(a) 레지스터 직접 주소 지정 (b) 직접 주소 지정

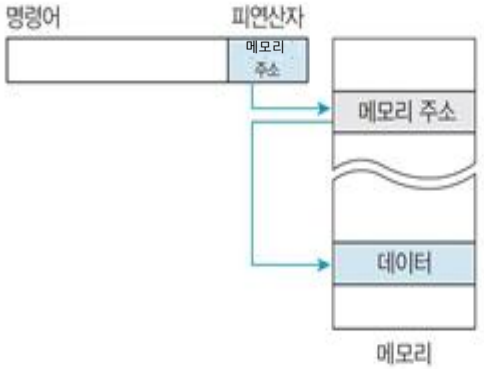
2.4.3 주소지정 방식

■ 단계별 분류로 정리한 주소 지정 방식 - 계속

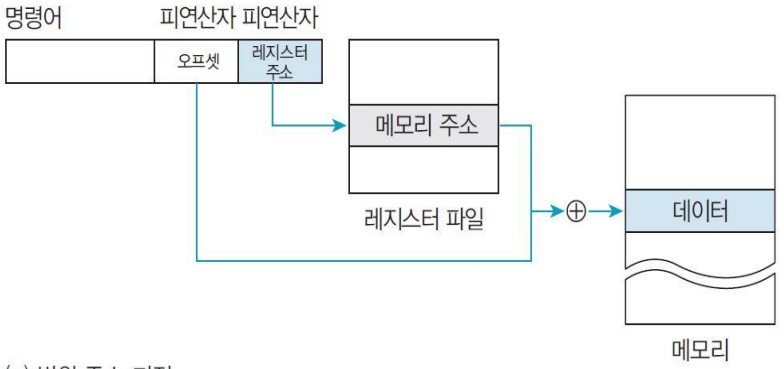
3) 2-단계 주소 지정



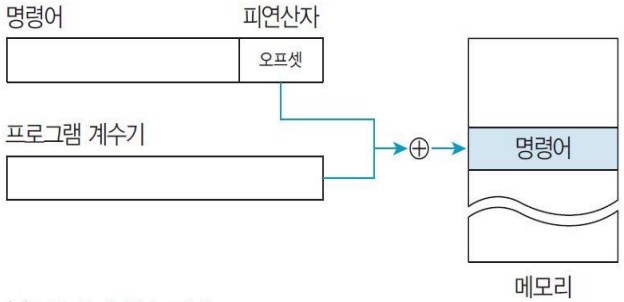
(a) 레지스터 간접 주소 지정



(b) 메모리 간접 주소 지정



(c) 변위 주소 지정



(d) PC 상대 주소 지정

2.4.3 주소지정 방식

■ 주소지정 방식 정리

- ✓ [1] load R1, #1000 ; R1 ← 1000 // 즉시(즉치)
- ✓ [2] load R1, 1000 ; R1 ← M[1000] // 메모리 직접
- ✓ [3] load R1, (1000) ; R1 ← M[M[1000]] // 메모리 간접
- ✓ [4] load R1, R2 ; R1 ← R2 // 레지스터 직접
- ✓ [5] load R1, (R2) ; R1 ← M[R2] // 레지스터 간접
- ✓ [6] NOP, PUSH, POP ; // 묵시
- ✓ [7] load R1, 6000(R2) ; R1 ← R1 + M[M[R2]+6000] // 변위
- ✓ [8] load R1, X(PC) ; R1 ← M[PC + X] // 상대
- ✓ [9] load R1, X(R_{index}) ; R1 ← M[R_{indx} + X] // 인덱스
- ✓ [10] load R1, X(R_{base}) ; R1 ← M[R_{base} + X] // 베이스

2.4.3 주소지정 방식

■ 주소지정 방식 실습

- ✓ 다음과 같은 레지스터와 메모리 주소에 대한 데이터 값이 있다. 각 명령어 실행 후의 R1의 값은?
- ✓ 명령어의 길이는 1바이트 고정 크기이며 PC는 인출사이클 이후 자동으로 증가함
- ✓ 명령어가 100번지 부터 연속적으로 있는 것이 아니라 각 명령어가 100번지에 위치해 있는 경우이다.
- ✓ 초기 값은 R1 = 100, R2 = 200, R_{auto} = 100, PC = 100 이며 두번째 오퍼랜드의 결과가 R1으로 저장됨

1)	MOV R1, #100 → 즉치	R1 = 100
2)	MOV R1, 100 → 메모리 직접	R1 = 201
3)	MOV R1, (100) → 메모리 간접	R1 = 70
4)	MOV R1, R2 → 레지스터 직접	R1 = 200
5)	MOV R1, (R2) → 레지스터 간접	R1 = 101
6)	MOV R1, 1(R2) → 변위	R1 = 70
7)	MOV R1, 1(PC) → 상대	R1 = 40
8)	LOAD R1, (R_{auto})+ → 인덱스	R1 = 201
	LOAD R1, (R_{auto})+	R1 = 199

메모리 주소	데이터
99	10
100	201
101	199
102	40
199	50
200	101
201	70

2.4.4 실제 상용 프로세서들의 명령어 형식

p126~p136

■ CISC(Complex Instruction Set Computer) 프로세서

- ✓ 명령어들의 수가 많음
- ✓ 명령어 길이가 일정하지 않음(명령어 종류에 따라 달라짐)
- ✓ 주소지정 방식이 매우 다양함
- ✓ 사용 예
  - ➢ PDP 계열 프로세서, Intel Pentium 계열 프로세서

■ RISC(Reduced Instruction Set Computer) 프로세서

- ✓ 명령어들의 수를 최소화
- ✓ 명령어 길이를 일정하게 고정 → 파이프라이닝 기법 적용에 매우 유리
- ✓ 주소지정 방식의 종류를 단순화
- ✓ 하버드 아키텍처
- ✓ 사용 예
  - ➢ ATmega microcontroller, ARM 계열 프로세서

2.4.4 실제 상용 프로세서들의 명령어 형식

■ RISC와 CISC의 전망

- ✓ CISC 구조의 명령어는 고급언어에 가까운 구조로서 하드웨어가 복잡함
- ✓ RISC 구조의 명령어는 기계어에 가까운 구조로서 소프트웨어가 복잡함
- ✓ 칩 밀도가 높아지고 하드웨어 속도가 빨라짐에 따라 RISC 구조와 CISC 구조의 경계가 붕괴
- ✓ 오늘날의 컴퓨터는 RISC 구조와 CISC 구조의 장점을 통합하는 추세 RISC와 CISC 특징 비교

■ RISC와 CISC 특징 비교

구분	RISC	CISC
명령어 형식	고정	가변
명령어 종류	적음	많음
명령어 길이	고정	가변
적재·저장 구조	사용	미사용
주소 지정 방식	단순하고 소수	복잡하고 다수
회로 구성	단순	복잡
장점	구현용이, 파이프라이닝에 효율적 적용	호환성 양호, 코드 밀도 양호
예	MIPS, ARM, PowerPC	Intel x86, DEC VAX 11/780

2.4.4 실제 상용 프로세서들의 명령어 형식

■ RISC의 특징 정리

1) 제한된 개수의 명령어 (128 혹은 그 이하)
2) 고정된 길이의 명령어
3) 클록 사이클당 하나의 명령어 실행
4) 제어 유니트 설계에 마이크로 프로그래밍 방식보다는 주로 고정배선 방식을 사용
5) 제한되고 간단한 어드레싱(주소지정) 모드 (최소 2개, 인덱스와 PC 상대)
6) 모든 동작이 레지스터에서 수행된다.
7) 메모리 동작은 적재와 저장의 2가지
8) 파이프라인 명령어 실행
9) 하버드 아키텍처
10) 많은 수의 범용 레지스터
11) 혹은 레지스터 사용 최적화 위한 고급 컴파일러 기술

2.4.4 실제 상용 프로세서들의 명령어 형식

생략

p126~p136

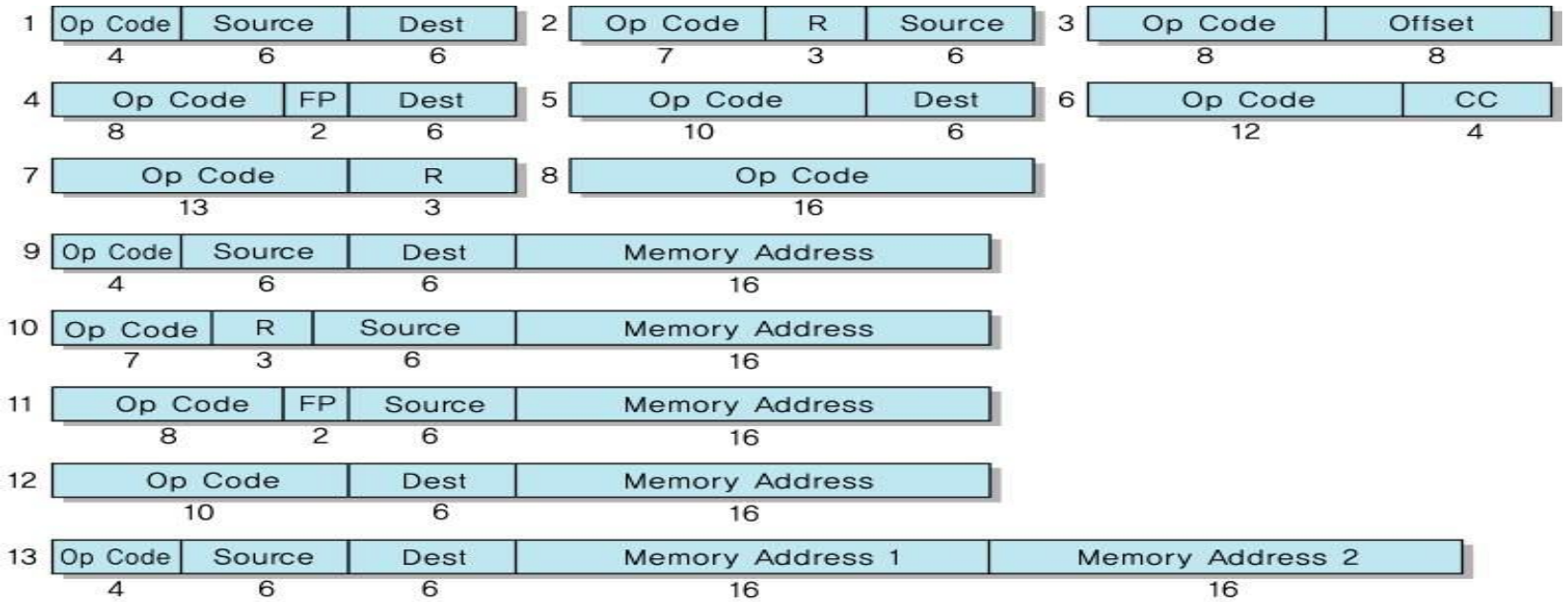
■ PDP-10 프로세서 : 고정 길이의 명령어 형식 사용

- ✓ 단어의 길이 = 36 비트, 명령어의 길이 = 36 비트
- ✓ 연산 코드 = 9 비트 → 최대 512 종류의 연산 허용 (실제 365 개)



■ PDP-11 프로세서 : 다양한 길이의 명령어 형식들 사용

- ✓ 연산 코드 = 4 ~ 16 비트
- ✓ 주소 개수 : 0, 1, 2 개



2.4.4 실제 상용 프로세서들의 명령어 형식

생략

p126~p136

■ 펜티엄 계열 프로세서의 명령어 형식

- ✓ 선형 주소(linear address: LA): 프로세서가 발생하는 주소
  = 유효 주소 + 세그먼트의 시작 주소
  - ➢ 각 세그먼트의 시작 주소는 해당 Segment Register에 저장
- ✓ 즉시 방식(immediate mode)
  - ➢ 데이터가 명령어에 포함되는 방식
  - ➢ 데이터의 길이 = 바이트, 단어(word) 혹은 2중 단어(double word)
- ✓ 레지스터 방식(register mode)
  - ➢ 유효 주소(EA)가 레지스터에 들어 있는 방식
- ✓ 변위 방식(displacement mode)
  - ➢ 명령어에 포함된 변위(주소)가 유효 주소로 사용되는 방식으로서, 직접 주소지정 방식에 해당
- ✓ 베이스 방식(base mode)
  - ➢ 레지스터 간접 주소지정에 해당
- ✓ 상대 방식(relative mode)
  - ➢ 변위값과 프로그램 카운터의 값을 더하여 다음 명령어의 주소로 사용하는 방식

주소지정 방식	유효 주소(EA)	선형 주소(LA)
즉시 방식	데이터 = A	
레지스터 방식	EA = R	LA = R
변위 방식	EA = A	LA = (SR) + EA
베이스 방식	EA = (BR)	LA = (SR) + EA
변위를 가진 베이스 방식	EA = (BR) + A	LA = (SR) + EA
변위를 가진 인덱스 방식	EA = (IX) + A	LA = (SR) + EA
인덱스와 변위를 가진 베이스 방식	EA = (IX) + (BR) + A	LA = (SR) + EA
상대 방식	EA = (PC) + A	LA = EA

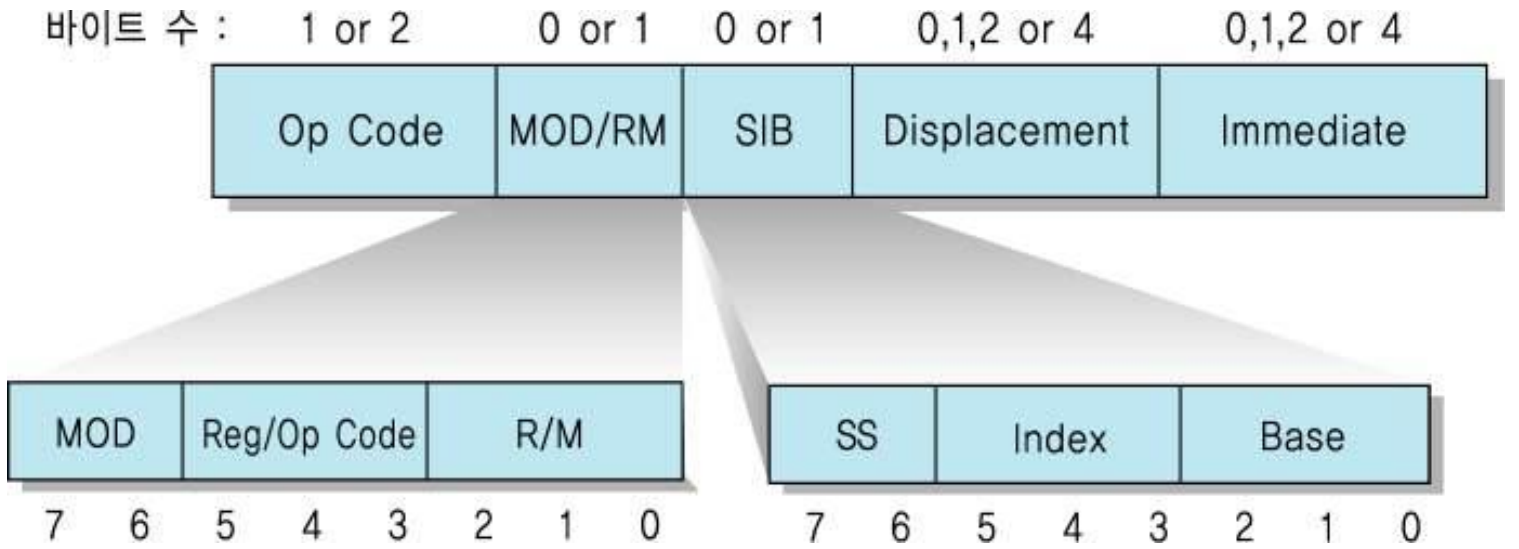
2.4.4 실제 상용 프로세서들의 명령어 형식

생략

p126~p136

■ 펜티엄 계열 명령어 형식의 필드들

- ✓ 연산 코드(Op code) : 연산의 종류 지정. 길이 = 1 or 2 바이트
- ✓ MOD/RM : 주소지정 방식 지정
- ✓ SIB : MOD/RM 필드와 결합하여 주소지정 방식을 완성
- ✓ 변위(displacement) : 부호화된 정수(변위)
- ✓ 즉시(immediate) : 즉시 데이터



2.4.4 실제 상용 프로세서들의 명령어 형식

p126~p136

■ ATmega Microcontroller의 명령어 형식

✓ ATmega128 microcontroller의 CPU 코어

- 8-비트 CPU
- 대부분 명령어들의 길이 : 16비트
- 기억장치 액세스 명령어들(LOAD, STORE) : 32비트
- 내부 레지스터 : 32개
- RISC 프로세서
- 특징
  - 연산 코드의 비트 수가 명령어에 따라 달라짐
  - 오퍼랜드의 위치가 유동적

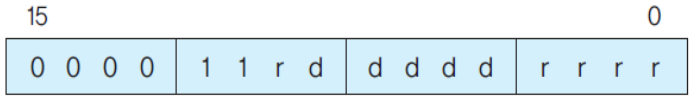
2.4.4 실제 상용 프로세서들의 명령어 형식

p126~p136

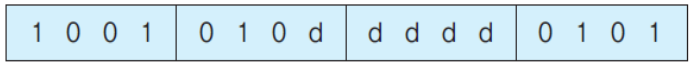
■ ATmega Microcontroller의 명령어 형식

✓ [명령어 형식의 예] 그림 2-35

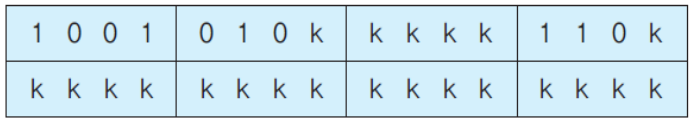
- (a) ADD Rd,Rr ; Rd ← Rd + Rr
- (b) ASR Rd ; 레지스터 Rd에 대하여 산술적 우측 시프트 수행
- (c) JMP K ; K 번지로 무조건 점프
- (d) BRVS K ; V 플래그가 세트 되었다면, K 번지로 분기
- (f) LDS Rd,K ; K 번지의 내용을 읽어서 Rd에 적재



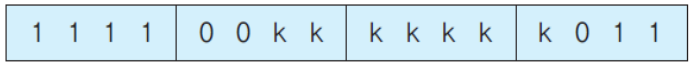
(a) ADD Rd,Rr 명령어



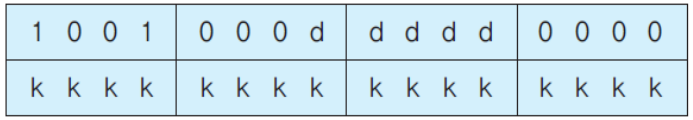
(b) ASR Rd 명령어



(c) JMP K 명령어



(d) BRVS K 명령어



(e) LDS Rd,K 명령어

2.4.4 실제 상용 프로세서들의 명령어 형식

p126~p136

■ ARM(Advanced RISC Machine) 계열 프로세서

- ✓ 32-비트 RISC 프로세서
- ✓ 개방형 아키텍처(Open Architecture)
- ✓ 모바일 시스템용 프로세서로 널리 사용
- ✓ 명령어 필드들
  - ➢ 분기조건 필드, 연산 필드, 오퍼랜드 필드, 등
  - ➢ 조건 플래그: N, Z, C, V
  - ➢ 레지스터 필드: Rn, Rd, Rs
  - ➢ P, U, W 비트: 주소지정 방식 결정
  - ➢ B 비트: 연산처리 단위 결정 (바이트(B=1) 혹은 단어(B=0))

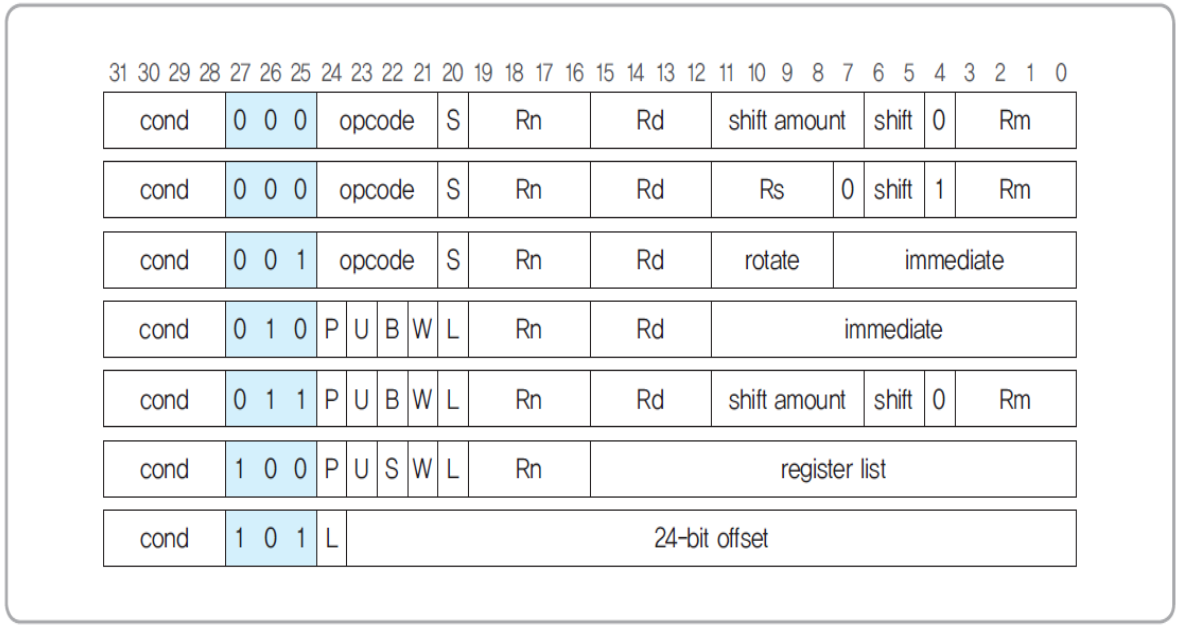


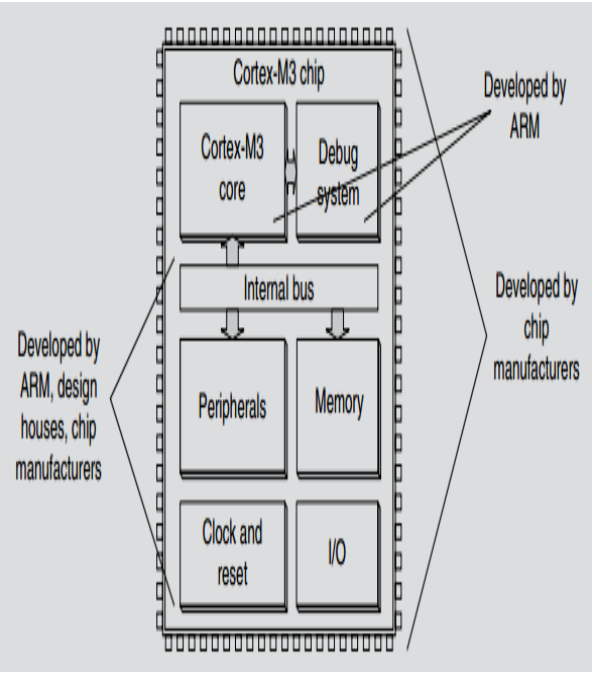
그림 2-36 ARM 프로세서의 명령어 형식들

- ✓ Thumb 명령어
  - ➢ 컴파일 과정에서 원래의 32-비트 명령어들을 분석하여 새로운 패턴의 16-비트 명령어 형식으로 축소 → 프로그램 기억장치 용량 감소 (각종 임베디드 시스템에서 널리 사용)

추가: ARM 프로세서

■ ARM 프로세서 개요

- ✓ 1983년 Arcon사에서 Berkley RISC-I을 변형하여 ARM CPU 개발
- ✓ 이후에 Arcon Computers와 Apple computer가 합작하여 Advanced RISC Machines Ltd 설립
- ✓ 다양한 종류의 RISC CPU를 지속적으로 개발
- ✓ 성능 보다는 제어 등의 산업용 특성에 맞게 **속도와 비용을 중시하는 변형된 RISC 구조**
  - ➢ Exception 처리 강화, 소형화, 집적화, 저전력화에 집중
- ✓ ARM사는 Core만 개발, 생산은 별도 라이선스 받은 업체
- ✓ 라이선스를 받은 업체들은 자기들이 필요로 하는 각종 주변장치 회로를 내장 시켜서 1-chip 형태의 마이크로 컨트롤러를 생산
  - ➢ Atmel, 브로드컴, 시러스 로직, 프리스케일, 퀄컴, 후지쯔, 인텔, 인피니언, IBM, OKI, 닌텐도, 삼성, 샤프, ST , VLSI, TI 등등...



추가: ARM 프로세서

■ ARM 프로세서 주요 특징

- ✓ 단순화 설계를 통한 코어 최소화 및 저전력용 구조 → Embedded System 최적화
- ✓ 단순성이 강조한 변형된 RISC 구조
  - ➢ 가변 Cycle 명령어 존재(메모리 블록전송), Register Window를 줄인 Shadow Register 사용 등
- ✓ 하위 명령어 구조를 유지 하면서 발전 (ISA)
  - ➢ 32비트 워드, 기본 명령어가 32비트, Thumb(16bit), DSP, Java 확장 명령어
- ✓ Pipeline (+ branch prediction)
- ✓ 데이터 바이트 배열
  - ➢ 리틀-엔디안(little-endian), 빅-엔디안(big-endian) 모두 지원
- ✓ 주소공간
  - ➢ 주소 32비트(4G 바이트), Memory-mapped I/O, 다양한 Addressing 모드
- ✓ 레지스터 구성 (32bit 길이, 총 37개)
  - ➢ 범용 레지스터 30개, PC 1개, CPSR 1개, SPSR 5개, Load-Store 구조
- ✓ 프로세서 내부에 하드웨어 디버그 기술 포함
- ✓ 동작모드 → 비특권모드(일반적인 사용자모드) 1개, 특권모드 6개

추가: ARM 프로세서

■ ARM 아키텍처, ARM (프로세서) 코어, ARM 프로세서

✓ **ARM 아키텍처 :**

- 명령어, 레지스터구조, 처리되는 데이터 크기 등과 같은 기본구성 및 동작원리
- ISA(Instruction Set Architecture)가 기본이 됨.
- v4, v5, v6, v7, v8

✓ **ARM 코어 (Core)**

- ARM 아키텍처의 기본원리를 이용해 만든 프로세서 코어
- ARM920T, ARM926EJ-S, ARM1026EJ-S, ARM1156T2(F)-S, Cortex-M1, Cortex-A72 등

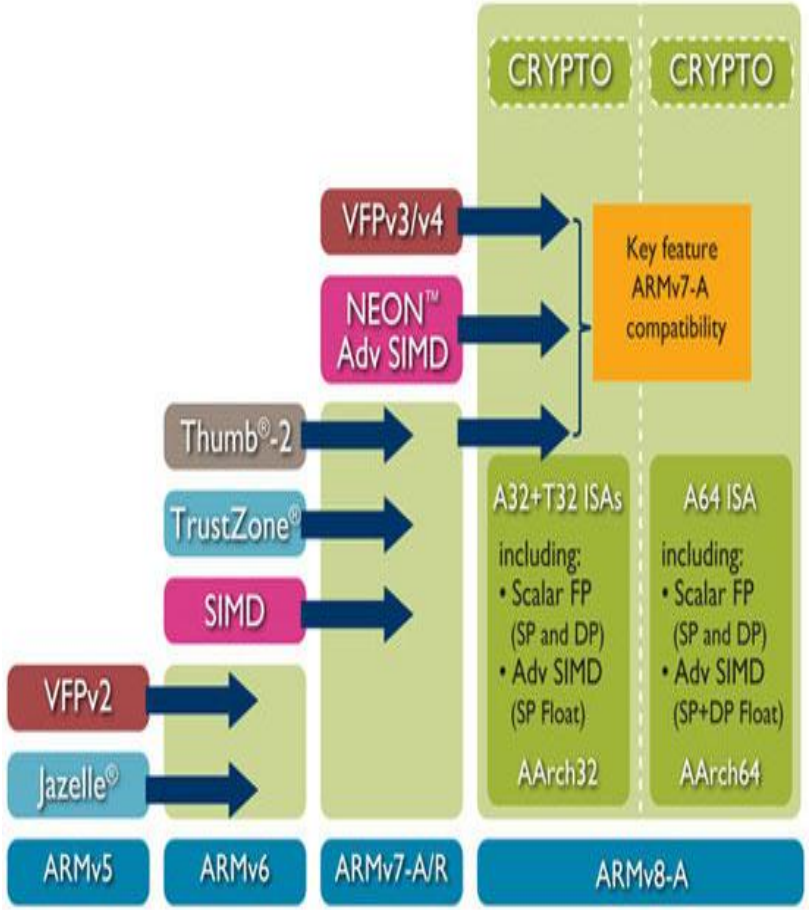
✓ **ARM 프로세서**

- ARM 코어에 주변회로를 포함하는 독립된 형태
- 각 반도체 업체마다 생산하는 SoC

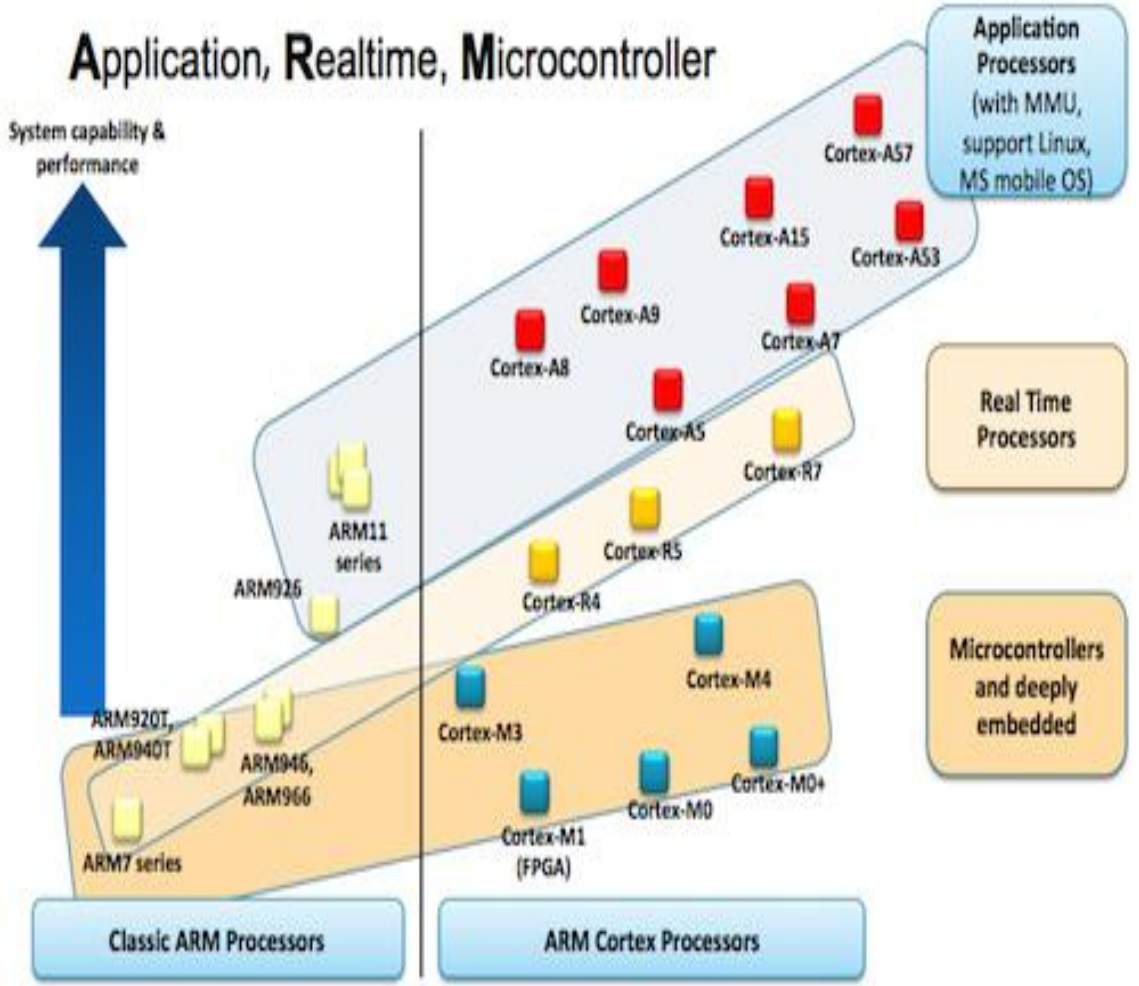
✓ https://en.wikipedia.org/wiki/ARM_architecture
✓ https://ko.wikipedia.org/wiki/ARM_아키텍처

추가: ARM 프로세서

■ ARM 아키텍처 변천사

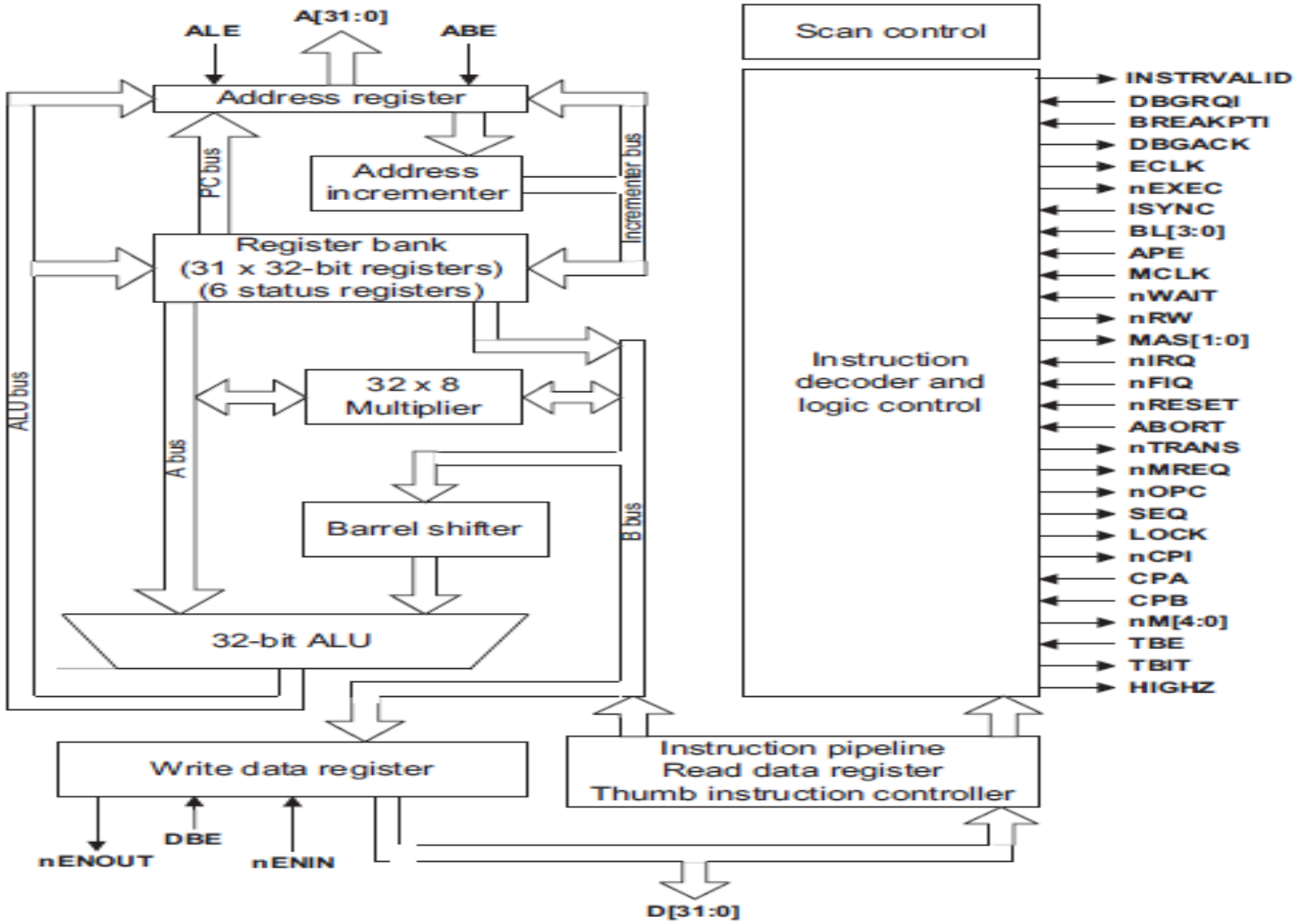


■ ARM 프로세서 코어 변천사



추가: ARM 프로세서

- **ARM 코어 (ARM7TDMI main processor logic)**



추가: ARM 프로세서

■ 주요 ARM (코어) 계열 비교

	ARM7	ARM9	ARM10	ARM11
Pipeline stages	3	5	6	8
동작속도(MHz)	80	150	260	335
소비전력 (mW/MHz)	0.06	0.19 (+cache)	0.5 (+cache)	0.4(+cache)
MIPS/MHz	0.97	1.1	1.3	1.2
Architecture	Von-Neumann	Harvard	Harvard	Harvard
Multiplier	8x32	8x32	16x32	16x32

■ ARM 명령어 (ISA)

- ✓ ARM instruction (32bits)
- ✓ Thumb instruction (16bits) → 코드 사이즈 (65% 수준 감소), 속도, 저전력
- ✓ Jazelle instruction (Java extension) → Java H/W 가속기

추가: ARM 프로세서

■ ARM 범용 레지스터 (ARM9)

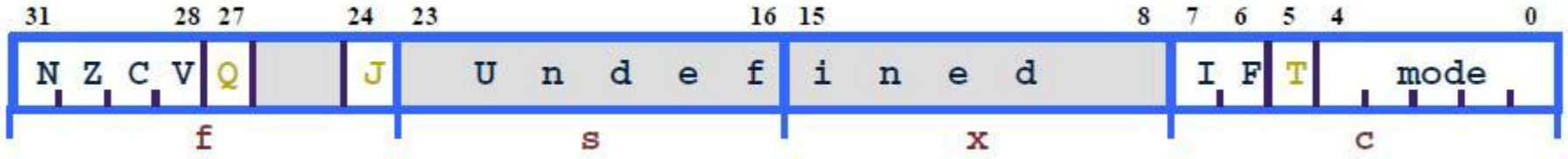
1) 32비트 30개의 범용 레지스터
   - ➢ R0~R15가 프로세서 모드에 따라 선택되어 사용
   - ➢ Thumb 상태에서는 R8~R15는 일반적으로 사용하지 않는다
2) 32비트 1개 PC 레지스터
3) 6개의 프로그램 상태 레지스터
   - ➢ CPSR(Current Program Status Register) 1개
   - ➢ SPSR (Saved Program Status Register) 5개
   - ➢ 특별한 명령어로만 접근 가능
4) 특별한 의미를 가지는 범용 레지스터
   - ➢ R13 = SP(Stack Pointer)
   - ➢ R14 = LR(Link Register)
   - ➢ R15 = PC(Program Counter) 32비트

User/ System	Supervisor	Abort	Undefined	Interrupt	Fast interrupt
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8_FIQ
R9	R9	R9	R9	R9	R9 _FIQ
R10	R10	R10	R10	R10	R10 _FIQ
R11	R11	R11	R11	R11	R11 _FIQ
R12	R12	R12	R12	R12	R12 _FIQ
R13 (SP)	R13_SVC	R13_ABORT	R13_UNDEF	R13_IRQ	R13 _FIQ
R14 (LR)	R14_SVC	R14_ABORT	R14_UNDEF	R14_IRQ	R14 _FIQ
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	SPSR_SVC	SPSR_ABORT	SPSR_UNDEF	SPSR_IRQ	SPSR_FIQ

추가: ARM 프로세서

■ ARM(ARM9) CPSR (Current program Status Register)



플래그	플래그명	1로 세트되는 경우
Q	Saturation	오버플로우나 포화가 발생하는 경우
V	oVerflow	Signed 오버플로우가 발생하는 경우
C	Carry	Unsigned 캐리 발생하는 경우
Z	Zero	결과가 0 인 경우, 종종 동일함을 표현하기 위해 사용
N	Negative	결과의 31번째 비트가 1인 경우

mode	M[4:0]
usr	10000
fiq	10001
irq	10010
svc	10011
abt	10111
und	11011
sys	11111

- ✓ 인터럽트 Disable 비트
  - ➢ I = 1, IRQ를 disable함
  - ➢ F= 1, FIQ를 disable함
- ✓ T 비트: '0'이면 명령어가 ARM 명령어이고 '1' 이면 Thumb 명령어를 표시
- ✓ mode 비트: 프로세서의 7개 모드 중 하나를 표시

추가: ARM 프로세서

■ ARM 명령어 분류

1) 산술 명령어
2) 논리 명령어
3) 시프트 명령어
4) 비교 명령어
5) 이동 명령어
6) Load/Store 명령어
7) 분기 명령어

모든 CPU들이 이런 종류의 명령어들을 가지고 있다

■ ARM 명령어 서식

✓ 데이터 처리 명령 형태

```
OP-code      Rd, Rn, Rx
```

➢ Rd= 대상 레지스터(저장), Rn = 오퍼랜드1(소스 레지스터), Rx = 오퍼랜드2

✓ Load/Store 명령 형태 (LDR, STR 명령어)

```
OP-code      Rd, [ Rn, Rx ]
```

➢ Rd = 대상 레지스터, Rn = 베이스 레지스터, Rx = 옵셋

추가: ARM 프로세서

■ ARM (32bit) 주요 명령어

✓ 산술 명령어 - 산술연산

ADD	32비트 값 덧셈	Rd=Rn + N
ADC	32비트 값과 캐리 덧셈	Rd=Rn + N + Carry
SUB	32비트 값 뺄셈	Rd=Rn - N
SBC	32비트 값과 캐리 뺄셈	Rd=Rn – N !(Carry)
RSB	32비트 값 반대로 뺄셈	Rd=N - Rn
RSC	32비트 값과 캐리 반대로 뺄셈	Rd=N – Rn - !(carry)

✓ 논리 명령어 & 시프트 명령어

AND	32비트 값 비트별 논리 AND	Rd=Rn & N
ORR	32비트 값 비트별 논리 OR	Rd=Rn | N
EOR	32비트 값 비트별 논리 EOR(XOR)	Rd=Rn ^ N
BIC	논리 비트 클리어(AND NOT)	Rd=Rn & ~N
LSL	논리 왼쪽 시프트	x << y
LSR	논리 오른쪽 시프트	x >> y
ASL	산술 왼쪽 시프트	(부호) x << y
ASR	산술 오른쪽 시프트	(부호) x >> y
ROR	오른쪽 회전	(x>>y) | (x<<(32-y))
RRX	확장 오른쪽 회전	(c<<31) | (x>>1)

✓ 비교 명령어

CMP	비교	Rn+N 결과 플래그 설정
CMN	반전 후 비교	Rn-N 결과 플래그 설정
TST	32비트 값 비트별 테스트	Rn ^N 결과 플래그 설정
TEQ	32비트 값 일치 검사	Rn &N 결과 플래그 설정

✓ 이동 명령어

MOV	32비트 값을 레지스터로 이동	Rn=N
MVN	32비트 값 반전 후 레지스터로 이동	Rn=~N

✓ LOAD/STORE 명령어

- RISC 프로세서의 특징인 Load-store 구조를 갖는다
- 레지스터/메모리와의 데이터 이동과 관련된 명령어들
- 즉 연산에 필요한 메모리들의 값을 미리 CPU 내부 레지스터로 옮긴 후 레지스터 끼리 연산을 하고 결과만 메모리로 저장 → 메모리 접근 횟수의 최소화(속도향상)

LDR	워드를 레지스터로 적재	Rd ← mem32[addr]
STR	레지스터 값을 저장	Rd → mem32[addr]
LDRH	반워드를 레지스터로 적재	Rd ← mem16[addr]
STRH	레지스터 반워드를 저장	Rd → mem16[addr]
LDRSH	유부호 반워드를 레지스터로 적재	Rd ← 부호확장(mem16[addr])
LDRB	바이트를 레지스터로 적재	Rd ← mem8[addr]
STRB	레지스터 바이트를 저장	Rd → mem8[addr]
ADR	레지스터에 어드레스 값 설정	Rd=addr

추가: ARM 프로세서

■ ARM (32bit) 주요 명령어

✓ ARM 제어 이동

- ➢ **분기(Branch) 명령어(B*xx*)** 사용하며 분기주소는 PC-상대주소(Relative)를 사용
- ➢ 상대주소 오프셋은 워드(32bit)로 표현되고 주소는 byte이므로 실제주소는 분기명령 오프셋 * 4
  - ❖ 예 : B #100 ; 현재의 PC + 400 번지로 점프

✓ 조건 코드 (B*xx*)

EQ	영과 일치	Z=1
NE	영과 불일치	Z=0
CS	캐리 설정	C=1
CC	캐리 클리어	C=0
MI	음수	N=1
PL	음이 아님	N=0
VS	오버플로우	V=1
VC	오버플로우 아님	V=0
HI	무부호 큼	C=1 이고 Z=0
LS	무부호 같거나 작음	C=0 또는 Z=1
GE	유부호 크거나 같음	N=V
LT	유부호 작음	N≠V
GT	유부호 큼	Z=0 이고 N=V
LE	유부호 작거나 같음	Z=1 또는 N≠V

추가: ARM 프로세서

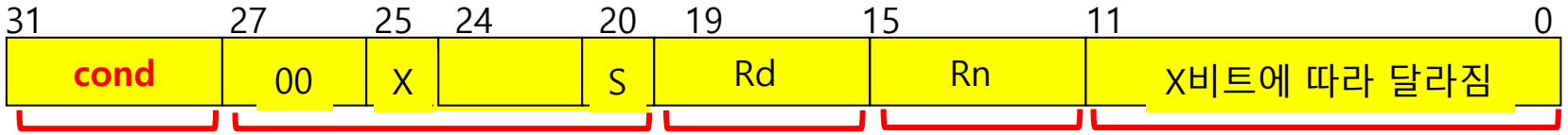
■ ARM 모드(32bit) 상세 명령어 형식

명령어 32비트 구성

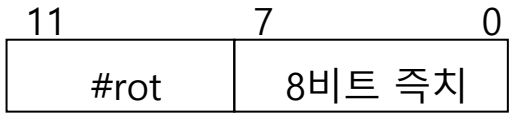
→ **Condition(4) + Opcode(8) + Destination(4) + Source1(4) + Source2(12)**

→ cond : condition 4bit (조건종류)

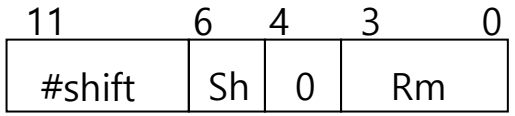
- 조건부 실행 → 분기명령을 배제 해서 파이프라인 기능을 최대화 한다

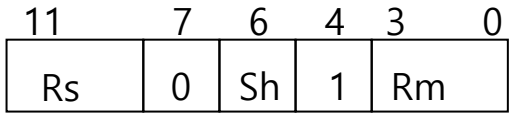


x=1



x=0





ADD Rd, Rn, Op2 // Rd = Rn + Op2
Rd = destination register
Rn = operand 1 (항상 register임)
Op2 = operand 2 (register 혹은 immediate 값)

cond
- condition 4bit (조건종류)
- 조건부 실행
- 이 기능을 이용해 분기명령을 배제 해서 파이프라인 기능을 최대화 한다

S : Set Condition Code
0 = unset, 1 = set

추가: ARM 프로세서

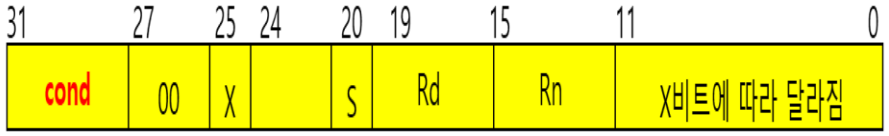
■ ARM 조건부 실행의 장점

```
//Trivial C code
if (a>b)
{
      ++a;
}
else
{
      ++b;
}
```

분기명령을 사용하지 않고
조건부실행명령을 통해 명령어가
5개에서 3개로 줄었다
- Code 축소 효과
- 파이프라인 극대화

```
//Use conditional execution only on branch
instructions → 분기명령을 사용
; a in reg v1
; b in reg v2
              cmp v1, v2
              ble INCB              //jump if a<=b
              add v1, v1, #1        //++a
              b DONECOND            //jump to done
INCB:
              add v2, v2, #1 ; ++b
DONECOND:
...
```

```
//Use conditional execution on regular instructions →
분기명령을 사용하지 않는다
; a in reg v1
; b in reg v2
              cmp v1, v2
              addgt v1, v1, #1      //++a if a>b
              addle  v2, v2, #1     //else ++b
...
```



조건부 실행. 이 기능을 이용해 분기명령을 배제 해서 파이프라인 기능을 최대화

추가: AVR 마이크로컨트롤러

■ AVR 마이크로컨트롤러의 특징





- ✓ 고성능의 RISC 구조
  - ➢ 명령어의 수와 주소지정방식을 최소화 함
  - ➢ 기본 명령의 실행시간이 1클록
  - ➢ 32개 범용 레지스터가 ALU에 직접 연결되어 모든 연산 및 전송이 레지스터간에 수행
  - ➢ 2단계의 파이프라인 구조
- ✓ 하버드 구조
  - ➢ 별도의 프로그램 버스와 데이터 버스로 동시 접근이 가능
- ✓ CMOS 구조
  - ➢ 소비전력이 적고, 동작 전압 범위가 큼
- ✓ 고성능 고효율의 플래시 메모리 탑재
  - ➢ 프로그램 메모리가 플래시 메모리로 되어 있음
- ✓ C언어에 최적화된 설계
  - ➢ RISC 구조의 CPU는 내부 명령어의 종류가 적기 때문에 작성한 프로그램을 컴파일 과정에서 얼마나 최적화 했는지 여부가 성능에 많은 영향을 미침

추가: AVR 마이크로컨트롤러

■ AVR 마이크로컨트롤러의 종류

	타이니	메가	X-메가
프로그램 메모리(KB)	1~8	4~256	32~384
핀 수(개)	6~32	28~100	44~100
지원하는 어셈블리 명령어 수(개)	~120	~135	~142
최대 동작 속도(MHz)	20	20	32

1) ATtiny Family
   - ➢ 8비트 범용 마이크로컨트롤러
   - ➢ 비교적 간단한 제품의 개발에 유용
   - ➢ 0.5KB~16KB의 플래시 메모리 내장

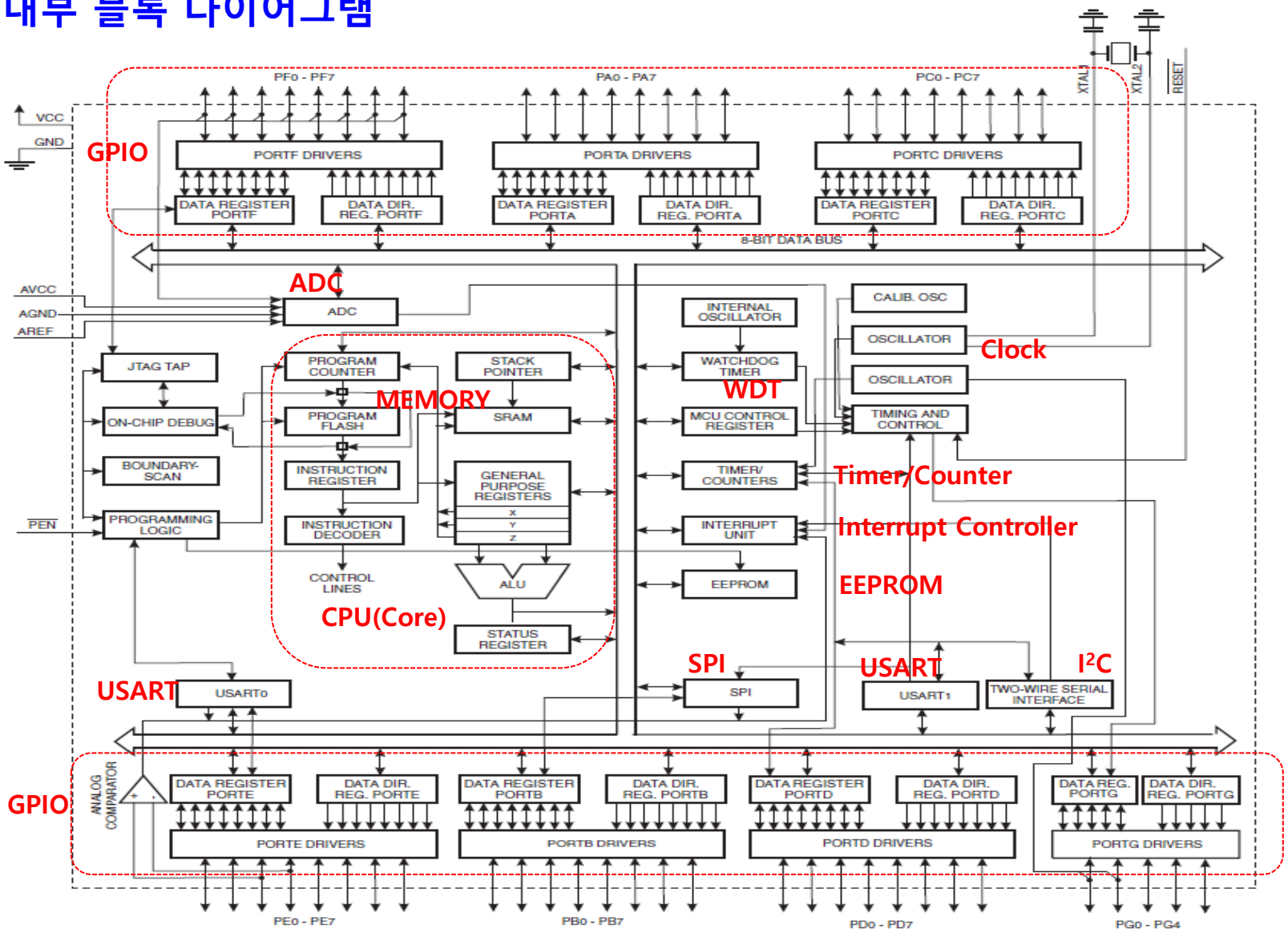
2) ATmega Family
   - ➢ 4KB~256KB의 플래시 메모리 내장
   - ➢ 핀의 수가 20~100핀 정도로, 최대 12개의 I/O포트를 가짐
   - ➢ 규모가 크며 성능이 높아, 큰 제품의 컨트롤러로 적당함

3) ATXmega Family
   - ➢ 8비트 범용 마이크로컨트롤러, 8KB~384KB의 플래시 메모리 내장.
   - ➢ ATmega 계열 이후에 좀 더 강력한 성능을 가짐.

4) 특수 용도
   - ➢ Automotive AVR(차량용), CAN AVR(CAN 통신용 기능 내장), LCD AVR(LCD 드라이버 내장), USB AVR(USB 기능 내장) 등

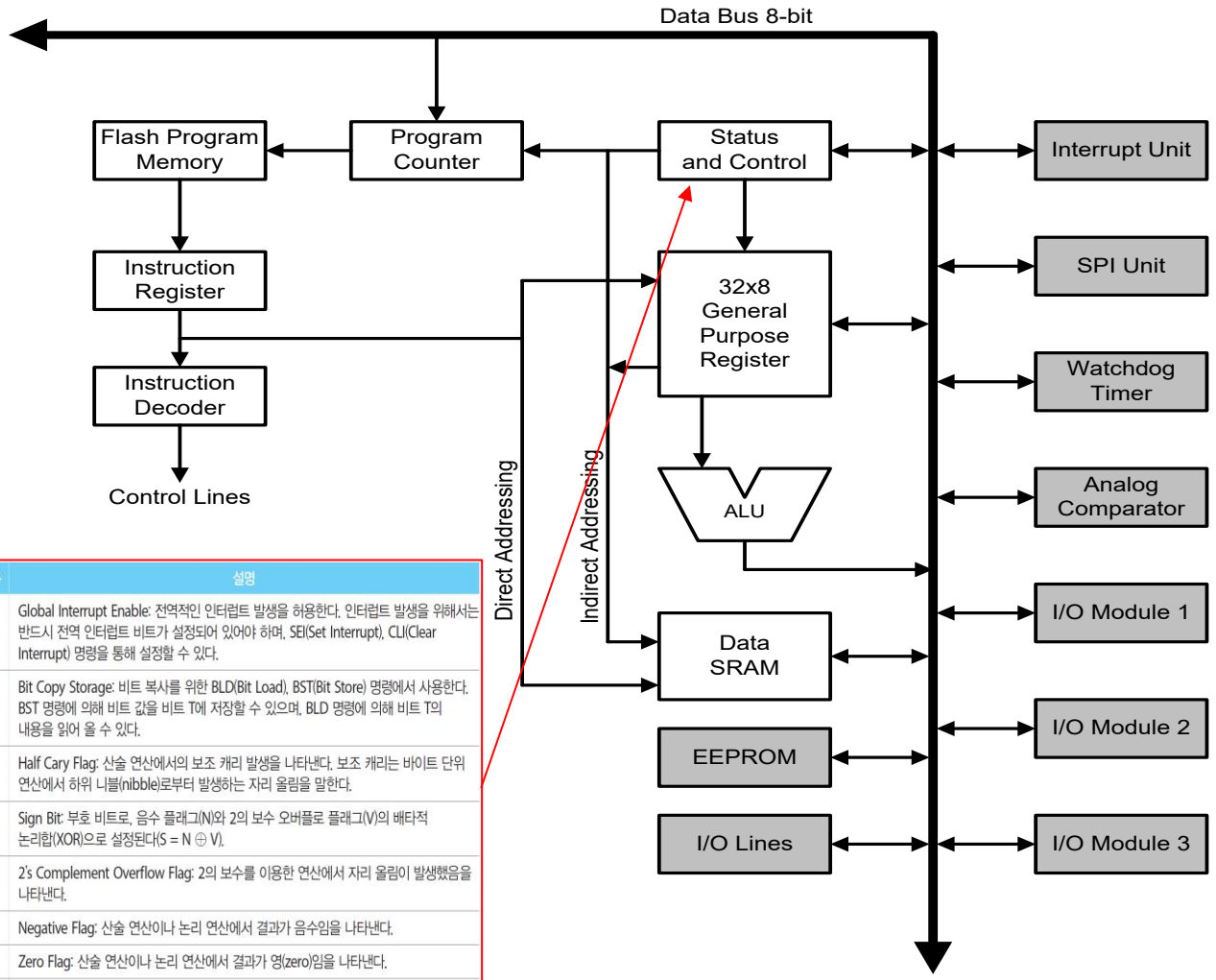
추가: AVR 마이크로컨트롤러

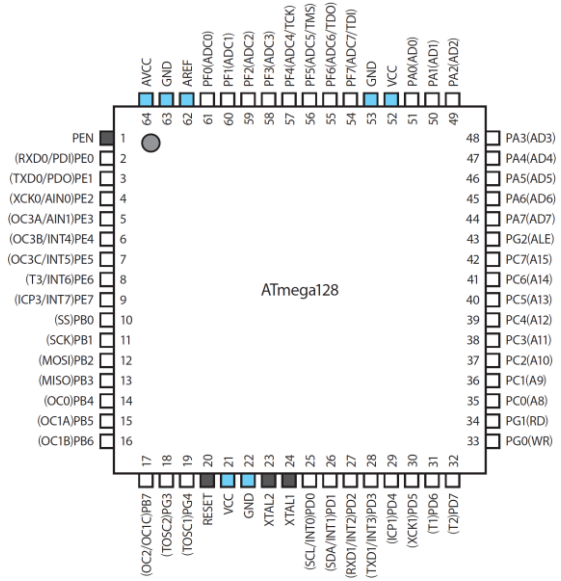
- **ATmega128의 내부 블록 다이어그램**

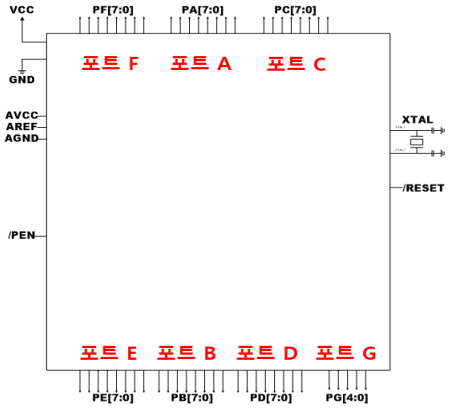


추가: AVR 마이크로컨트롤러

■ ATmega128 코어(Core)를 확대한 모습 및 핀 배열







비트 번호	비트 이름	설명
7	I	Global Interrupt Enable: 전역적인 인터럽트 발생을 허용한다. 인터럽트 발생을 위해서는 반드시 전역 인터럽트 비트가 설정되어 있어야 하며, SEI(Set Interrupt), CLI(Clear Interrupt) 명령을 통해 설정할 수 있다.
6	T	Bit Copy Storage: 비트 복사를 위한 BLD(Bit Load), BST(Bit Store) 명령에서 사용한다. BST 명령에 의해 비트 값을 비트 T에 저장할 수 있으며, BLD 명령에 의해 비트 T의 내용을 읽어 올 수 있다.
5	H	Half Cary Flag: 산술 연산에서의 보조 캐리 발생을 나타낸다. 보조 캐리는 바이트 단위 연산에서 하위 니블(nibble)로부터 발생하는 자리 올림을 말한다.
4	S	Sign Bit: 부호 비트로, 음수 플래그(N)와 2의 보수 오버플로 플래그(V)의 배타적 논리합(XOR)으로 설정된다(S = N ⊕ V).
3	V	2's Complement Overflow Flag: 2의 보수를 이용한 연산에서 자리 올림이 발생했음을 나타낸다.
2	N	Negative Flag: 산술 연산이나 논리 연산에서 결과가 음수임을 나타낸다.
1	Z	Zero Flag: 산술 연산이나 논리 연산에서 결과가 영(zero)임을 나타낸다.
0	C	Carry Flag: 산술 연산이나 논리 연산에서 캐리가 발생했음을 나타낸다.

추가: AVR 마이크로컨트롤러

■ ATmega128의 특징 - 1/2

- ✓ 다양한 메가 시리즈 중 ATmega128은 적당한 핀 크기로 인해 가장 많이 사용 → 쉽게 구할 수 있다
- ✓ 고성능, 저전력의 AVR 8비트 마이크로컨트롤러
- ✓ 진보된 RISC(Reduced Instruction Set Computer) 구조
  - ➢ 대부분 단일 클록에서 실행되는 강력한 133개의 명령어 → 16MHz에서 거의 16MIPS로 동작됨
  - ➢ 8 bit의 32개로 구성된 범용 작업 레지스터와 추가된 주변장치 제어 레지스터
  - ➢ 2 사이클 내에서 수행되는 강력한 곱셈기를 내장
- ✓ JTAG(IEEE 표준 1149.1 호환) 인터페이스 지원
  - ➢ JTAG 표준에 따르는 바운더리 스캔(Boundary-Scan)과 호환
  - ➢ 온- 칩 디버그(On-Chip Debug)지원 및 JTAG 인터페이스를 통한 플래시, EEPROM, 퓨즈, 락 비트 설정
- ✓ 메모리 (Memory)
  - ➢ 프로그램을 읽고 쓸 수 있는 128KB의 플래시 프로그램(Flash Program) 메모리 내장
  - ➢ 4Kbytes의 EEPROM 내장
  - ➢ 4Kbytes의 SRAM 내장

추가: AVR 마이크로컨트롤러

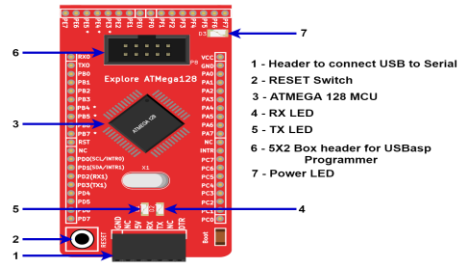
■ ATmega128의 특징 - 1/2

✓ 주변 장치 (Peripheral)

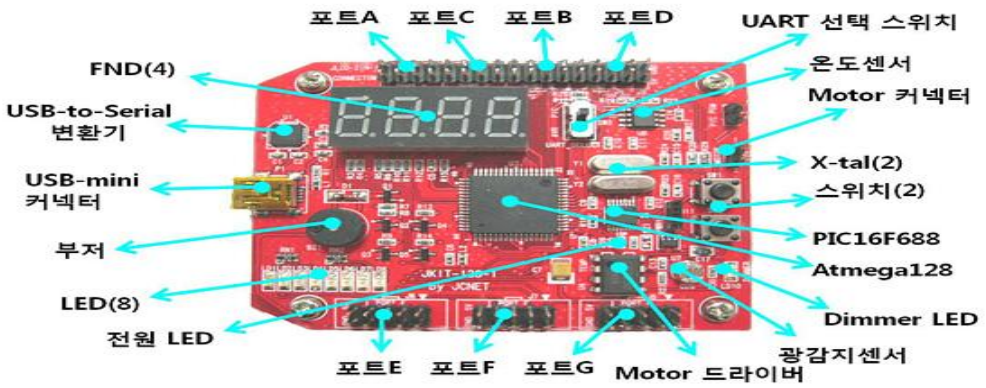
- 각각 2개씩의 8bit, 16비트 타이머/카운터 지원
- 외부 오실레이터에 의한 실시간 카운터(RTC)
- 6개의 PWM(Pulse With Modulation) 채널
- 8채널, 10bit의 ADC 내장
- TWI(Two-Wired Interface) 통신 지원
- 프로그래밍 가능한 직렬 USART 지원
- SPI(Serial Peripheral Interface) 지원
- 프로그래밍 가능한 워치독(Watchdog) 타이머 지원
- 아날로그 비교기(Analog Comparator) 지원

✓ 특수 기능

- 파워-온 리셋(POR)과 브라운 아웃(Brown out) 검출
- RC 오실레이터에 의한 내부 발진 지원
- 외부/내부 인터럽트 소스 지원
- 6개의 슬립 모드(Sleep Mode) 지원





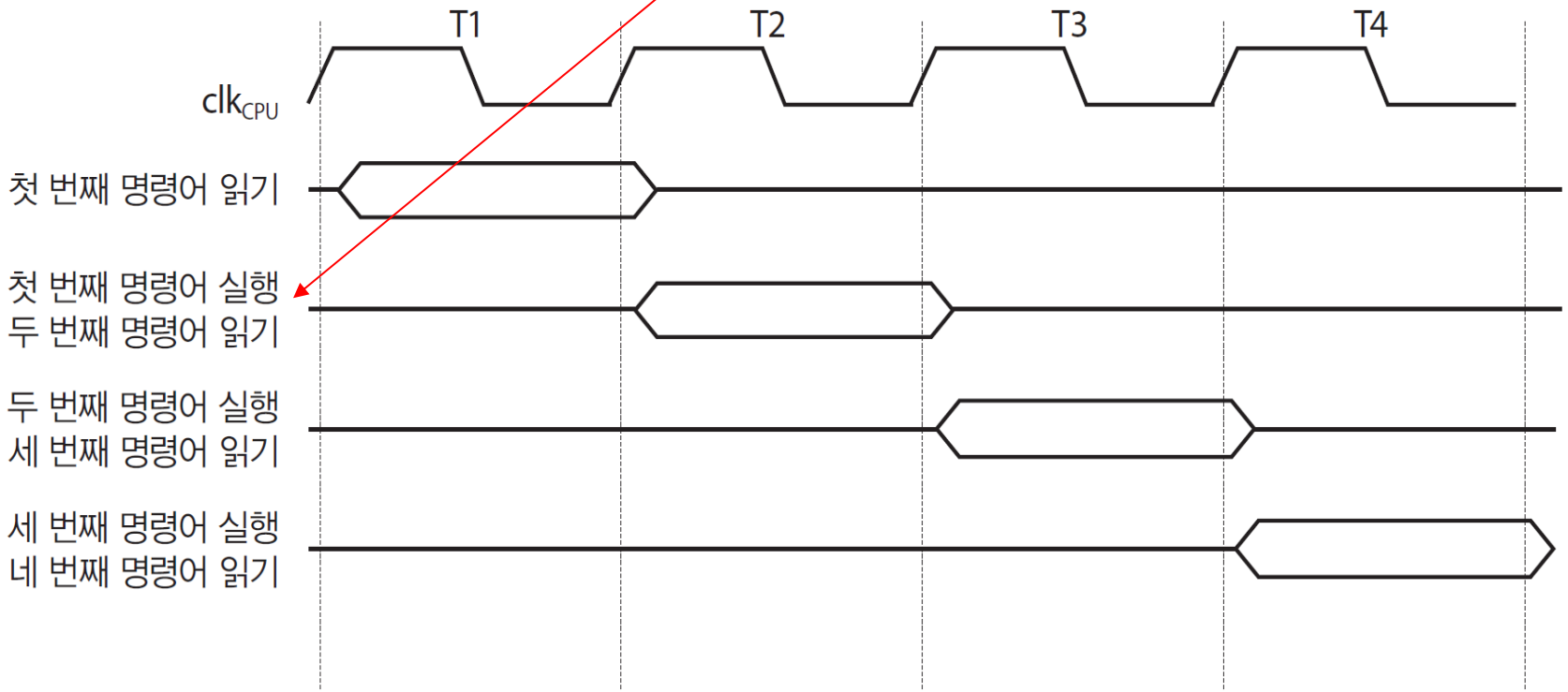


추가: AVR 마이크로컨트롤러

■ 명령어의 실행 – 1/2

✓ **①명령어 읽기(인출)**와 **②명령어 실행**의 **2단계가 파이프라인**에 의해 중첩되어 실행

- 1클록에 1개 명령어 실행 가능
- RISC 구조이므로 가능

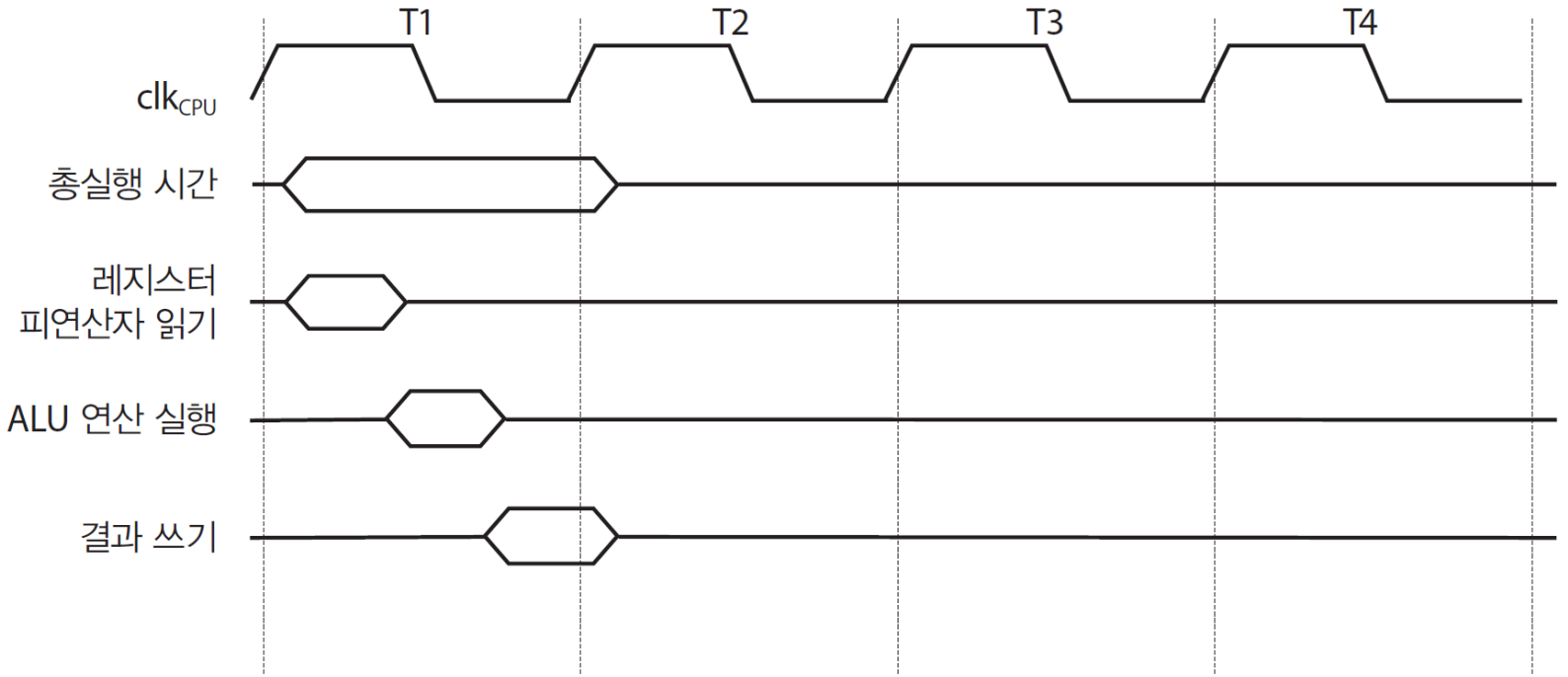


추가: AVR 마이크로컨트롤러

■ 명령어의 실행 – 2/2

✓ CPU 내에서의 명령어 실행은 3단계의 세부 단계로 나뉘어 진행

➢ **①레지스터/피연산자 읽기 → ②ALU 내의 연산 → ③결과 쓰기**



Thank you